Highlights

cert-machine: independent three-valued certification of machine-proposed numerical claims in exact and interval arithmetic[⋆]

Carlos Toledo, Pedro Veras Guimarães

- Machine-proposed numbers are decided by a checker that shares no code with the claimant.
- Three verdicts, each published with the object that justifies it; a refusal names its reason.
- Soundness proved under the inclusion property; every function of the arithmetic table carries a stated tail bound.
- A pre-registered benchmark of 572 claims of known truth: no false certification, no false refutation; two defects found and fixed.
- Exact certificates re-check without the engine; interval ones re-decide from pinned inputs.

cert-machine: independent three-valued certification of machine-proposed numerical claims in exact and interval arithmetic

Carlos Toledo[a,*], Pedro Veras Guimarães[b]

[a]*Independent researcher, the cert-machine project, Brazil*
[b]*Department of Mechanical Engineering, Universidade Federal de Santa Catarina, Florianópolis, Brazil*

Abstract

Search programs, surrogate models and large language models now propose constants, counterexamples, fitted parameters and operating points faster than they can be refereed, each with a checker written by its producer. We present cert-machine, a system that decides such claims independently of the claimant. A claim is read as a conjunction of inequalities over a declared box of inputs, under a universal or an epistemic reading, and receives one of three verdicts—certified, refuted or refused—each published with the object that justifies it: a certificate with its falsifier, a falsifying input, or a refusal's reason and, where a parameter is declared, the threshold at which the verdict would change. Verdicts are proved sound under the inclusion property of the interval extensions the implementation computes, secured by exact rational arithmetic and outward-rounded IEEE-754 interval arithmetic with a stated tail bound for each function of its arithmetic table. Classical certifiers reduce analytic questions to finitely many such decisions, and a protocol of executable gates (pinned inputs, mandatory adversarial controls, second implementations, re-executed records) guards the code on which soundness rests. On a pre-registered benchmark of 572 claims of known truth, built by mutating 97 published objects of its public register with an exact wit-

*Draft v0.9 of 2026-10-27, built from repository commit 1eb3cd33c866; it will be archived with the next release of the repository, under the concept doi:10.5281/zenodo.22225860 (the previous version is in release v2026.10.21, doi:10.5281/zenodo.23551746). Every measured number and count is a macro read at build time from the records named in the Data availability statement, and a record that no longer supports the sentence quoting it stops the build. The supplementary material (S1–S5) holds the proofs, the function lemmas, the certifiers in detail, the decision loop and the reproduction table; the records and public reports of the use cases of Section 3.3 are in the repository archive [1], and the engineering demonstrator and the metocean study are [2] and [3].

*Corresponding author.

Email addresses: carlos@carlostoledo.co (Carlos Toledo), pedro.veras.guimaraes@ufsc.br (Pedro Veras Guimarães)

ness for each from a program sharing no code with the deciders, the system produced 0 false certifications and 0 false refutations, refused 6 false claims for resources, and exposed two defects of its own, fixed on the record. On 1,012,121 comparisons with an independent library no enclosure failed to contain the exact value, and on a register of 132 machine-generated claims its refutations expose gaps between what a producer's checker verified and what its output states. The contribution is the system and its semantics, not interval arithmetic, which is standard.

Keywords: interval arithmetic, validated numerics, computer-assisted proof, certificates, decision procedures, machine-generated mathematics, reproducibility

---

1. Introduction

Context. Machines now propose mathematics and engineering numbers at a rate no referee can follow. Conjectures on fundamental constants are generated from numerical coincidences [4]; constructions in combinatorics and analysis are found by program search guided by language models [5, 6, 7] and by learned search [8, 9, 10]; autonomous research systems publish record configurations with their own verification files [11]; long proofs arrive with machine formalisations [12, 13, 14]; and in engineering, surrogate models, optimisers and language models propose operating points and design values [2]. In every case the number reaches its reader together with a checker written by its producer.

Problem. A checker written by the producer of a claim is subject to the same modelling and implementation errors as the claim. The register of this project [15] records, among other defects, a floating-point product printed as an exact constant, witnesses that hold only within the verifier's own tolerance, a checker that compares in double precision what it encloses at a hundred digits, a pairwise condition proved by structure and never checked pair by pair, and a clause present in a paper's theorem but absent from the formal statement that was proved [16]. What is missing is a decider that reads the claim from its published statement and bytes, never runs the claimant's program in its trust path, and returns an answer that a third party can re-check without trusting the decider either.

Existing tools, and the gap. The ingredients of such a decider are decades old: interval arithmetic with outward rounding [17, 18, 19], the Krawczyk operator [20, 21], verification methods and ball arithmetic [22, 23], interval global optimisation [24, 25], filtered exact predicates [26, 27, 28], δ-complete decision procedures [29, 30], and certificates checked inside a proof assistant [31, 32, 33]. Each answers a question posed by its own user about

its own model (Section 4.3). None of them, to our knowledge, is organised around deciding *other parties'* published numbers: reading a printed digit string as the box it denotes, distinguishing a claim about every input in a box from a claim about an unknown true input inside it, and turning a failure to decide into a published result that states what would decide it. In the engineering instances of this paper the second, epistemic reading is the one asked, and neither verification and validation nor uncertainty quantification [34] answers it: the former qualifies a model, the latter gives a dispersion, whereas the engineer asks whether a given rule holds for every input that the declared evidence allows.

Objective. This paper asks three questions. (Q1) Can a single three-valued decision grammar, proved sound under one hypothesis, cover claims arriving from mathematics, from engineering and from metocean statistics? (Q2) What must the arithmetic and the software process guarantee for that hypothesis to hold in an implementation? (Q3) On claims whose truth is known, how often does the system certify, refute or refuse, and at what cost?

Contributions. The contribution is a system, not a numerical technique:

1. a three-valued decision grammar with witnesses, refusal as a result and flip thresholds, with a universal and an epistemic reading, and its soundness theorem (Section 2.2.2);
2. an arithmetic layer that secures the theorem's single hypothesis on IEEE-754 doubles and exact rationals, with a lemma for each elementary and special function it uses (Section 2.2.1, Appendix A);
3. a catalogue of standard certifiers, each mapped to the grammar with its witness and its refusal reasons (Section 2.2.4);
4. a verification protocol whose rules are executable gates rather than guidelines, with the gates that are not yet enforced listed as such (Section 2.4.2); and
5. a pre-registered evaluation of the system against claims of known truth, built by mutating its own public register with exact witnesses from programs sharing no code with the deciders, which found two defects of the system by its rule and measures what soundness promises (Section 2.3, Section 3).

Interval arithmetic, the Krawczyk operator, filtered predicates, sums of squares and Sturm sequences are cited, not claimed.

Outline. Section 2 presents the method in four layers: foundations, mathematical formulation, metrics and decision rules, and software architecture with its protocol. Section 3 reports the benchmark with known truth, the kernel's comparison with an independent library, the public register with its three largest exercises, and the coverage across domains; Section 4 counts

the independence the system can claim, places it beside existing tools, and states the threats to validity and the limits; Section 5 concludes. The supplementary material holds the proofs (S1, S2), the certifiers in detail (S3), the decision loop (S4) and the reproduction table (S5).

2. Methodology

The four layers run from the mathematics on which a verdict rests to the code that computes it. Four implementations recur below as running instances. (a) The *flow-line gate* of the engineering demonstrator [2] decides, under the epistemic reading, whether the wellhead pressure of a water-injection line stays below a rule over a declared envelope of roughness, viscosity, diameter, density, elevation and length, with Colebrook's friction law solved inside the enclosure; the pump pressure is its operating choice. (b) The *rock-physics engine* [35] decides, over declared boxes of rock and fluid properties, whether a time-lapse seismic change is guaranteed to be detectable. (c) The *likelihood certifier* of the metocean study [3] certifies each maximum-likelihood fit of an extreme-value family to a buoy's hourly record as the unique maximiser in its box, and ranks the families. (d) A *universal branch-and-bound* decides a statement for every input in a box by bisection on the widest coordinate; its instance in the register bounds, for every monic polynomial of a given degree with all roots in $[-1, 1]$, the measure of the set where the polynomial is below one in absolute value (one register row [15]).

2.1. Foundations

The system rests on three results. The first is the *fundamental theorem of interval arithmetic*: the interval evaluation of an expression encloses the range of the function it denotes [17, 36, 18], which turns finite floating-point computation into proof. The arithmetic is standardised as IEEE 1788-2015 [19, 37], test suites check conformance [38], and directed rounding can be emulated when the rounding mode cannot be set [39], the situation of the implementation described here (Proposition 1); the interval layer is not claimed conformant, and its operations were run on the standard's test suite (Section 3.2). The same arithmetic underlies verification methods [21, 22, 23], interval branch-and-bound [24, 25, 40, 41] and computer-assisted proofs from dynamics to the Kepler conjecture and partial differential equations [42, 43, 44, 45, 46]. Exact geometric computation [28] gives the same guarantee by another route, filtering a floating-point evaluation through a proved error bound and falling back to exact arithmetic when the filter fails [26, 27].

The second concerns what cannot be decided. The first-order theory of real closed fields is decidable [47], but with the elementary functions it is undecidable in general whether an expression is nonnegative everywhere [48], so a sound procedure over exp, log or sin must be allowed to return no

answer. δ-complete procedures weaken the question by a perturbation δ chosen by the user [29, 30]; interval constraint propagation returns boxes of unknown status [49]. Three-valued logic gives the undetermined answer a semantics [50]: a conjunction is false if any conjunct is false, undetermined if none is false and some is undetermined, and true otherwise. The verdict lattice of Section 2.2.2 adopts this operationally, with refusal as the undetermined value: a refusal records that the procedure did not decide, not that the claim lacks a truth value.

The third is the separation between finding an answer and checking it. A *certifying algorithm* returns, with each output, a witness from which a simple and independent checker can confirm the output without trusting the algorithm [51], as proof-carrying code [52], SAT solvers with replayable proofs [53, 54], rational sums of squares [55, 56] and Taylor-interval certificates replayed in a proof assistant [31, 57, 32] do. From statistical decision theory comes the last ingredient: a classifier that may abstain is evaluated by its risk on the cases it decides and by the fraction it decides [58, 59, 60, 61]. Test batteries sound by construction, whose reported defects are never spurious, have recently been built for language-model outputs [62]; cert-machine asks the same of every verdict, not only of refutations.

2.2. Mathematical formulation

2.2.1. Interval arithmetic and the inclusion property

Let $\mathbb{IR}$ denote the set of closed bounded real intervals $X = [\underline{x}, \overline{x}]$, with width $w(X) = \overline{x} - \underline{x}$ and midpoint $m(X) = (\underline{x} + \overline{x})/2$. A box is a vector $X = (X_1, \ldots, X_n) \in \mathbb{IR}^n$, with $w(X) = \max_i w(X_i)$. For $\circ \in \{+, -, \times, \div\}$ the interval operations are the set images $X \circ Y = \{x \circ y : x \in X,\ y \in Y\}$ ($0 \notin Y$ if $\circ = \div$), computed from the endpoints [36, 18]:

$$\begin{aligned} X + Y &= [\underline{x} + \underline{y},\ \overline{x} + \overline{y}], & X - Y &= [\underline{x} - \overline{y},\ \overline{x} - \underline{y}], \\ X \times Y &= [\min S,\ \max S],\ S = \{\underline{x}\underline{y}, \underline{x}\overline{y}, \overline{x}\underline{y}, \overline{x}\overline{y}\}, & X \div Y &= X \times [1/\overline{y},\ 1/\underline{y}]. \end{aligned} \tag{1}$$

Definition 1 (Interval extension)**.** Let $g : D \subseteq \mathbb{R}^n \to \mathbb{R}$. A map G from boxes in D to $\mathbb{IR}$ is an *interval extension* of g if $g(X) = \{g(x) : x \in X\} \subseteq G(X)$ for every box $X \subseteq D$ (the *inclusion property*), and *inclusion isotone* if $X \subseteq Y$ implies $G(X) \subseteq G(Y)$. This is Neumaier's inclusion function [18], weaker than Moore's extension, which also asks $G([x, x]) = g(x)$; nothing below needs more.

The *natural extension* of an expression replaces each real operation by its interval operation and each elementary function by an enclosure of its range.

Theorem 1 (Fundamental theorem of interval arithmetic [36, 18])**.** *If every elementary function occurring in an expression for g has an inclusion-isotone*

interval extension, then the natural extension G of g is an inclusion-isotone interval extension of g.

Theorem 1 is the only property that the soundness of a verdict requires (Proposition 2). Its price is overestimation. Writing $q(A, B) = \max(|\underline{a} - \underline{b}|, |\overline{a} - \overline{b}|)$ for the Hausdorff distance between intervals, an extension has *convergence order* [63] ω on B if there are $\kappa > 0$ and $\eta \geq 0$ with

$$q\big(g(X), G(X)\big) \leq \kappa\, w(X)^{\omega} + \eta \qquad \text{for every box } X \subseteq B. \tag{2}$$

In exact arithmetic $\eta = 0$; in floating point, outward rounding and the tail bounds of the function enclosures add a floor $\eta > 0$ that does not shrink with $w(X)$. The natural extension has order $\omega = 1$ when every elementary function has a Lipschitz extension, and the mean-value form $G_{\mathrm{MV}}(X) = G([c, c]) + \nabla G(X)^{\top}(X - c)$, $c \in X$, has order $\omega = 2$ when ∇G is a Lipschitz extension of the gradient [18]. cert-machine uses the natural and mean-value forms, endpoint evaluation of monotone functions, the Krawczyk operator and second-order Taylor remainders in quadrature; it uses no Taylor models [64] or affine forms, because the claims it meets are low-dimensional and either exact or decided with a margin that bisection reaches. The order governs how much subdivision a decision costs (Proposition 3), never whether the decision is sound.

Outward rounding on doubles. IEEE-754 arithmetic in round-to-nearest returns, for each of $+, -, \times, \div$, the double nearest the exact result [65, 66, 67], so evaluating Eq. (1) in round-to-nearest does not enclose the exact endpoints. ECMAScript specifies each of these operations on binary64 doubles with round-to-nearest and no fused multiply–add contraction or extended precision, so Proposition 1 presumes nothing beyond the standard; JavaScript cannot set the rounding mode, and directed rounding is emulated by moving each computed endpoint one unit in the last place outward [68, 39].

Proposition 1. *Let $\circ \in \{+, -, \times, \div\}$, let A, B be intervals of doubles—pairs of non-NaN doubles with lower end at most upper end, an infinite endpoint allowed as a bound—with $0 \notin B$ when $\circ = \div$, and let y_-, y_+ be the round-to-nearest values of the smallest and largest endpoint combinations $a \circ b$. If no endpoint combination is indeterminate ($0 \cdot \infty$, $\infty - \infty$, ∞/∞), then $A \circ B \subseteq [\mathrm{down}(y_-), \mathrm{up}(y_+)]$, where down and up step one double toward $\mp\infty$ and fix $\mp\infty$; an exact result beyond the largest double rounds to $\pm\infty$, which the step fixes, so overflow is enclosed.*

The proof is in Appendix A. The domain of Proposition 1 is enforced, not assumed: a malformed operand or an indeterminate result is refused by exception and never decided; division by an interval containing zero is

refused, not extended; a non-integer or overflowing integer exponent is refused. The trust base is the correctly rounded $+$, $-$, $\times$, $\div$ and square root that IEEE-754 mandates, the engine's arbitrary-precision integers, and nothing in the platform's mathematical library, which ECMAScript leaves "implementation-approximated"—with the one exception of Section 4.5: six modules, among them the radii-polynomial certifier, still call an older helper that pads the platform's cosine and sine by a margin that is derived, not proved. Where the quantity is rational or a tie must be decided, exact arithmetic is used instead: reduced fractions of arbitrary-precision integers, to which doubles convert losslessly, with exact elimination that reports a singular matrix rather than a small pivot.

Elementary and special functions. Theorem 1 requires an enclosure for every elementary function, and no library function is trusted as one. Each function f is evaluated as a truncated series S_N in interval arithmetic plus an explicit bound R_N on the omitted tail, so that $f(x) \in S_N(x) + [-R_N, R_N]$, the bound itself evaluated in interval arithmetic and its upper end added by an outward-rounded addition, so that nothing is absorbed into the widening of Proposition 1.

Assumption 1 (Function enclosures)**.** *For every function f of Table A.1 and every box X in its declared domain, either the call is refused or the returned interval $F(X)$ satisfies $f(X) \subseteq F(X)$, including near zeros of f.*

Table A.1 (Appendix A) lists each function with its reduction, series, tail bound and domain; the lemma that establishes Assumption 1 for it is stated and proved in S1. The kernel and its functions have been compared with Arb [23] on 1,012,121 seeded cases over 24 operations, with no enclosure that failed to contain the exact value (Section 3.2); that is evidence on sampled inputs, and the lemmas are the proof.

Long sums and derivatives. A running interval sum rounds outward at each of n additions, so its width grows linearly in n; at the 175,320 hourly values of one buoy it is about a thousand times what the terms carry. Each endpoint stream $p_1, \ldots, p_n$ is therefore summed by the compensated algorithm Sum2 of Ogita, Rump and Oishi [69, Proposition 4.5], whose result $\tilde{s}$ satisfies, also under underflow,

$$\Big|\tilde{s} - \sum_{i=1}^{n} p_i\Big| \le u \Big|\sum_{i=1}^{n} p_i\Big| + \gamma_{n-1}^2 \sum_{i=1}^{n} |p_i|, \qquad \gamma_k = \frac{ku}{1-ku}, \quad u = 2^{-53}, \quad (3)$$

provided $nu < 1$ and no overflow occurs; both hypotheses are checked by name and a stream that fails either is refused. The implementation evaluates the bound from computable quantities as $u|\tilde{s}| + 2\gamma_n^2 \sum |p_i|$, each step rounded upward, the doubled term covering the difference between $u|\tilde{s}|$ and

$u|\sum p_i|$ (S1). Gradients and Hessians come from second-order forward differentiation over the same interval arithmetic, or from univariate Taylor jets, so that the derivative a proof uses is an extension of the function the proof is about.

2.2.2. Claims, readings and verdicts

Boxes and claims. A box with rational endpoints is the object a claim is decided on; a double is a rational, so a box of doubles is such a box. A *declared envelope* B is a box each of whose sides carries the source of its bounds (a survey, a data sheet, a chart). A printed number is the box its digits allow: a decimal $\hat{d}$ printed with k fractional digits denotes

$$\left[\hat{d} - \tfrac{1}{2}10^{-k},\ \hat{d} + \tfrac{1}{2}10^{-k}\right], \tag{4}$$

so a printed 0.0634 is $[0.06335, 0.06345]$, both endpoints exact rationals. Equation (4) assumes that a printed *value* was rounded to nearest; the claim schema fixes that convention and refuses any other box for it. A printed *bound* ($\leq$, $<$, $\geq$, $>$) is taken as the exact rational its digits denote, and a value that was truncated or given to significant figures must be declared as an explicit rational box. Declared envelope ends are read as the nearest double, widened outward by one ulp unless the conversion is exact. One claim of Section 3.3 shows the whole path: a registry bound printed as $C_{1d} \leq 1.463071582607559$ is read as the exact rational of its digits; the certificate it rests on evaluates, in exact arithmetic, to $1.46307158260755932746\ldots$, which exceeds the printed bound, so the claim as printed is REFUTED with that value as its falsifier, and the bound 1.4630715826075594, the certificate's own value rounded up, is CERTIFIED beside it.

Definition 2 (Claim)**.** A *claim* is a pair (B, φ) of a domain B—a box, or a finite set of rational points, covered by its thin boxes—and a predicate

$$\varphi(x) \equiv \bigwedge_{j=1}^{m} g_j(x) \leq c_j \tag{5}$$

whose functions have interval extensions G_j; an equality, or an exact predicate—the orientation of three points, or a Boolean combination of exact integer tests such as the angular test of a kissing configuration—is the special case decided in exact arithmetic. It is read either *universally*, as $\forall x \in B : \varphi(x)$, or *epistemically*, as $\varphi(x^\dagger)$ for an unknown true input $x^\dagger \in B$.

The form is deliberately narrow: a conjunction of inequalities over one box, with disjunction allowed only among exact tests and no alternation of quantifiers. A statement outside it—an existential claim, an inequality quantified over a parameter that is itself unknown, a claim about an object that was not published—is not forced into the schema; it is refused with

the reason *the instrument declined*, or marked *needs data* with the bytes that would decide it named (Table 1). Under the universal reading a single proved counterexample refutes the claim; under the epistemic reading it shows only that the declared evidence does not settle the question.

Definition 3 (Verdicts)**.** Let $\mathcal{X}$ be a finite cover of B by boxes. The claim is *certified* if $\sup G_j(X) \le c_j$ for every j and every $X \in \mathcal{X}$; *refuted*, under the universal reading, if some box $X^* \subseteq B$ has $\inf G_j(X^*) > c_j$ for some j, and under the epistemic reading if every $X \in \mathcal{X}$ has $\inf G_j(X) > c_j$ for some j; and *refused* otherwise. An epistemic refusal is *split* when two points $x^-, x^+ \in B$ have been proved to violate and to satisfy φ, and *unresolved* otherwise.

Certification is the same test under both readings; the readings differ in what refutes—one proved violation, or a violation proved on all of B—and therefore in what a refusal leaves open.

Proposition 2 (Soundness)**.** *If every G_j is an interval extension of g_j, then* CERTIFIED *implies $\varphi(x)$ for every $x \in B$;* REFUTED *implies $\neg\varphi(x)$ for some $x \in B$ under the universal reading and for every $x \in B$ under the epistemic reading; and a split* REFUSED *implies that neither $\forall x \in B\, \varphi(x)$ nor $\forall x \in B\, \neg\varphi(x)$ holds, so that $\varphi(x^\dagger)$ is not determined by the declared envelope.*

Proof. Let $x \in B$ and $X \in \mathcal{X}$ contain it. By the inclusion property $g_j(x) \in g_j(X) \subseteq G_j(X)$, hence $g_j(x) \le \sup G_j(X) \le c_j$ for every j. The refutations follow by the same argument with the inequality reversed, on X^* or on every member of the cover. For a split refusal, x^+ and x^- are points of B at which φ is proved true and false; either could be $x^\dagger$. □

In the engineering gates the box decided is the double hull $B^+ \supseteq B$ obtained by widening each declared end outward; certification and epistemic refutation transfer from B^+ to B, and the witness of a split refusal is a corner of B itself, the declared decimal enclosed by its two neighbouring doubles and decided as that thin box. A witness that a floating-point search proposes (the rock-physics engine) is a double of B^+, within one ulp of B, and is reported as such.

Soundness says nothing about how often the procedure decides. The following result, in the spirit of δ-completeness [29], states when bisection is guaranteed to certify and what it costs; the proof is in S2.

Proposition 3 (Termination for robust claims)**.** *Let each G_j have convergence order ω with constants κ and η on B (Eq. (2)), and let the claim hold with margin $\varepsilon > \eta$: $g_j(x) \le c_j - \varepsilon$ for every j and every $x \in B$. Then every box $X \subseteq B$ with $w(X) \le ((\varepsilon - \eta)/\kappa)^{1/\omega}$ is certified, and bisection of the*

widest coordinate certifies the claim at depth at most

$$d^* = n\left\lceil \log_2\Big(w(B)\,\big(\kappa/(\varepsilon-\eta)\big)^{1/\omega}\Big)\right\rceil^+, \tag{6}$$

with at most 2^{d^*} *boxes in the final cover, where* $\lceil t\rceil^+ = \max(0, \lceil t\rceil)$.

Proposition 3 explains the refusals the evaluation must expect. A claim whose margin does not exceed the floor ($\varepsilon \le \eta$; in particular a bound attained at a point) is never certified by bisection alone and needs one of the certifiers of Section 2.2.4 or a higher precision that lowers η; and the box count grows as $(w(B)(\kappa/(\varepsilon-\eta))^{1/\omega})^n$, so a depth budget turns high-dimensional or nearly tight claims into refusals by budget rather than into wrong verdicts. Read in the contrapositive, a refusal by budget at depth $d_{\max}$ states that the claim is not robust by more than $\delta = \eta + \kappa\,(2^{-d_{\max}/n} w(B))^{\omega}$, the counterpart of a δ-satisfiable answer. The bound is proved for widest-coordinate bisection; the engineering gates split by sensitivity instead (Table 3), for which no depth bound is claimed.

Composition. Order the verdicts as REFUTED $\prec$ REFUSED $\prec$ CERTIFIED. A record with several checks $\varphi = \bigwedge_i \varphi_i$ receives

$$v(\varphi) = \min_{\prec}\{v(\varphi_i)\}, \tag{7}$$

that is, REFUTED if any check is refuted, otherwise REFUSED if any is refused, otherwise CERTIFIED—the strong conjunction of Kleene's three-valued logic [50].

Corollary 1. *Under Proposition 2, if all checks* φ_i *are decided on the same box* B *and under the same reading, the composition rule Eq.* (7) *is sound for* φ *under that reading (proof in S2).*

A split refusal does not survive conjunction: if φ_1 holds only on one half of B and φ_2 only on the other, each is split while $\varphi_1 \wedge \varphi_2$ is false on all of B. The implementation composes from the checks' words alone, so a composed refusal is never labelled split; split refusals are issued by the two engineering gates at the level of one check, a single inequality. Items of one record decided on different objects are not composed but reported as MIXED with the counts of each verdict (Table 1). A refusal is recorded permanently and is reopened only by a new exact argument—a finer cover, an extension of higher order, another certifier, or the bytes the claimant had not yet published; no probabilistic grading is attached to any verdict.

Witnesses. Every verdict carries the object that justifies it, in the sense of a certifying algorithm [51]. A certification carries its *certificate*: the inputs pinned by digest, the hull of its enclosures with the counts of its cover, and a statement of what would make it fail (the *falsifier*); the constructor

refuses a certificate without a stated falsifier and a refusal without a reason. A refutation carries the falsifying object, printed; a split refusal its two witnesses, decided as thin boxes by the same enclosures. An exact certificate re-checks without the system's code; an interval certificate records the hull and the counts of its cover, not the cover, and is re-checked by re-deciding the claim from the pinned inputs. The public register composes the three verdicts into words that say what they are made of (Table 1); none is a fourth verdict, and the kinds of refusal (instrument declined, claimant published nothing, budget exhausted, box did not close, audit reach stated) are never summed into one total.

Table 1: Verdicts and their compositions in the public register [15].

word	composed of	witness carried
CERTIFIED	—	certificate with its falsifier; an exact one re-checks without the engine
REFUTED	—	falsifying object, printed
REFUSED	—	reason; for a split refusal, two witnesses and the thresholds
PARTIAL	certified + refused	which fragment certified and which core was not reached
NEEDS DATA	refused, cause with claimant	the bytes that would decide the row
MIXED	certified + refuted, one record	counts of both and the items refuted
REPAIRED	refuted + certified	object as printed refuted; object beside it certified
MISPRINTED	refuted, kind *not from the published data*	the printed value and the exact value the bytes give
UNWITNESSED	refused	the bytes witness no value for the printed claim
QUEUED	not a verdict	nothing; not counted as decided

2.2.3. Flip thresholds

A split refusal leaves open a question that has a size. Let the claim depend on a declared scalar p—an operating choice, or one end of an interval that a measurement could narrow—through boxes $B(p)$ and predicates φ_p, and write $\mathsf{T}(p)$ for "$\varphi_p(x)$ holds for every $x \in B(p)$" and $\mathsf{F}(p)$ for "$\varphi_p(x)$ fails for every $x \in B(p)$".

Assumption 2 (Monotone truth)**.** $\mathsf{T}(p)$ *implies* $\mathsf{T}(p')$ *for every* $p' \le p$*, and* $\mathsf{F}(p)$ *implies* $\mathsf{F}(p')$ *for every* $p' \ge p$*. A sufficient condition is that the boxes do not depend on* p *and every* $g_j(x;p)$ *is nondecreasing in* p*.*

Proposition 4. *Let Assumption 2 and the hypothesis of Proposition 2 hold, let* $B(p)$ *be nonempty, and define* $p_T = \sup\{p : \mathsf{T}(p)\}$ *and* $p_F = \inf\{p : \mathsf{F}(p)\}$*, so that* $p_T \le p_F$*. If the procedure, at a value* m *of the parameter, certifies*

the claim, then $\mathsf{T}(p)$ *holds for every* $p \le m$ *and* $p_T \ge m$*; if it refutes it epistemically, then* $\mathsf{F}(p)$ *holds for every* $p \ge m$ *and* $p_F \le m$*; if it returns a split refusal, then* $p_T \le m \le p_F$*.*

The proof, the closed form where p enters additively, the second case—a measurement that narrows one side of the box, where nested boxes replace the monotone-truth assumption—and the way each running instance verifies the hypotheses are in S2. The threshold turns a refusal into a measurement or an operating change with a known effect on the verdict: at this pump pressure the rule is proved, and with the roughness shown to be at least this value it is proved too [2].

2.2.4. Certifiers

Each certifier turns a question that is analytic in form into finitely many decisions of the kind Proposition 2 covers (Table 2), and most exist for the claims that Proposition 3 cannot reach: zeros, maxima and bounds attained with no margin. None is new; the contribution is that each is mapped to the grammar with an explicit witness and explicit refusal reasons. S3 states the result each rests on and its implementation.

Proposers. Every certifier takes its candidate from a floating-point search—Nelder–Mead and Levenberg–Marquardt for a likelihood, Newton for a root, sampling for a minimum, a language model for a construction—and none of these is in the trust path. A proposer is not assumed correct; provided Assumption 1 holds, the certifier implements its theorem correctly and the certifier is the only writer of the verdict (Table 4 lists two exceptions), a proposer can cost a refusal, never a false verdict.

2.3. Comparison metrics and decision rules

2.3.1. Metrics

Verdict–truth confusion matrix. Each benchmark claim $i = 1, \dots, N$ carries a ground-truth label $\tau_i \in \{\text{true}, \text{false}, \text{malformed}\}$—it holds on all of B, it fails at a proved witness, or it is not a well-formed instance of its claim schema—established by an oracle that shares no code with cert-machine and is itself exact (Section 2.3.3). It receives a verdict $v_i \in \{\textsc{certified}, \textsc{refuted}, \textsc{refused}\}$, a refusal being split or unresolved. The confusion matrix $N_{v\tau} = \#\{i : v_i = v,\ \tau_i = \tau\}$ is the primary result. Proposition 2 predicts structural zeros in the cells that would be errors:

$$N_{\textsc{certified},\text{false}} = 0, \quad N_{\textsc{refuted},\text{true}} = 0, \quad N_{\textsc{certified},\text{malf.}} = N_{\textsc{refuted},\text{malf.}} = 0, \tag{8}$$

the last pair because a malformed object must be refused by its shape check and never decided. Any nonzero entry in Eq. (8) is a defect in code or in a hypothesis, and is reported individually. Under the epistemic reading a fourth label, *undetermined* (the predicate holds at one proved point of B and

Table 2: Certifiers, the result each rests on, the witness it publishes and the reasons for which it refuses (details in S3). For a formal proof, the statement that the proof proves is read against the theorem claimed, not decided.

question	result used	witness	refusal reasons
a finite set, exhaustively	every element decided; a Cauchy–Schwarz bound, asserted on the data before any path runs, makes the floating-point products exact	counts; any violating element	set beyond the budget; bound not met
an entropy certificate	exact pushforwards; each $-p \ln p$ enclosed	enclosure of the ratio	enclosure straddles the bar; malformed law
a zero, exactly one	Krawczyk operator [20, 21]; radii polynomial for fixed points [70]	$K(X) \subseteq \operatorname{int} X$; or $p(r) < 0$ and contraction factor < 1	no contraction within the round limit or below the radius cap
a maximum in a box	strict concavity: unique critical point and a Hessian proved negative definite over the box	Hessian minors or nonsingularity proof	no contraction; Hessian not proved negative definite; climb stopped at the family's boundary
a supremum at a limit	mean value theorem on the score in the limiting coordinate	neighbourhood, score sign	sign not proved
every input in a box	bisection (Proposition 3); monotone implicit roots; multilinear corners	cover; corner witnesses	budget exhausted (split or unresolved)
a sign, exactly	static filter, exact fallback [26, 27]	sign, or ON	literal beyond fixed precision
a polynomial ≥ 0 (no register row; no proposer wired)	rational SOS identity [55, 56]	q_i, $c_i \geq 0$, zero residual	negative weight; nonzero residual (printed)
min of a cosine polynomial	Chebyshev form, Sturm chain, interval Newton, Markov bound	enclosure of the minimum; algebraic certificate	straddles the bar; isolation or refinement fails; algebraic: minimum not interior
a formal proof, built	kernel acceptance at the pinned toolchain, replayed by a second kernel [71, 72]	build log; axioms reported	resources (stack, memory, time)

fails at another), has the split refusal as its correct answer; the benchmark of Section 3 contains universal and exact claims only, so that label does not arise there.

Upper bound on the error rate. Observing zero errors on N_{F} false claims does not show that the error probability is zero. The one-sided Clopper–Pearson bound [73] at level $1-\beta$ after x errors is the beta quantile $\theta_U = \mathrm{Beta}^{-1}(1-\beta;\ x+1,\ N_{\mathrm{F}}-x)$, which at $x=0$ is $1-\beta^{1/N_{\mathrm{F}}} \approx 3/N_{\mathrm{F}}$ for $\beta = 0.05$ [74]; the same bound is reported for false refutations on true claims. The bound presumes independent draws from a population of claims, whereas the benchmark is assembled from chosen records and injected faults, and a defect in one code path affects every claim of its class; it is read as a description of this benchmark, not as an error rate that transfers. Proposition 2 is what the system claims mathematically; the bound measures the gap between the code and the theorem on the claims tested.

Coverage and selective risk. A sound three-valued decider is a selective classifier with a reject option [58, 59, 60], in which a split refusal is an informative answer and only an unresolved refusal is an abstention. Writing D for the number of informative answers, its *coverage* is $C = D/N$ and its *selective risk* R is the number of errors of Eq. (8) divided by D. Because the mix of labels is set by the benchmark's design, C is also reported per label—the certification rate on true claims, the refutation rate on false claims and the refusal rate on malformed ones—with the unresolved refusals broken down by reason and never summed across reasons. Soundness predicts $R = 0$; the measured R tests the implementation against that prediction, and the performance of a sound implementation is then carried by C. A floating-point checker is placed on the same risk–coverage plane [59, 61]: it has $C = 1$ and $R \geq 0$; cert-machine is predicted to have $C \leq 1$ and $R = 0$.

Cost and agreement. For a split refusal the thresholds of Section 2.2.3 say how far an operating change or a measurement must go. The harness records wall-clock time, CPU time and peak memory for each run, under two profiles (a portable one with a wall-clock cap, an extended one without); Section 3.1 reports the campaign's totals. Verdicts are deterministic, so agreement with a second implementation is exact agreement, field by field or by digest, and a single disagreement is a defect; against an established library the primary result is the table of containment per operation.

2.3.2. Decision rules

Table 3 lists every choice the system makes when it decides a claim, with the rule and the result that justifies it. The first two rows are modelling choices, declared with the claim: they fix *what* is decided, and a different choice decides a different claim. Every other rule either is forced by a result above or can change only the cost of a decision, never its soundness.

Table 3: Decision rules and their mathematical basis.

decision	rule	basis
reading a printed number	a value: the box of half a unit in the last printed place; a bound: the exact rational of its digits	Eq. (4)
universal or epistemic	universal if the claim quantifies over inputs; epistemic if it concerns one unknown true input	Definition 2
arithmetic	exact rationals if the object is rational or a tie must be decided; outward-rounded intervals otherwise	Proposition 1, Theorem 1
extension	natural form (engineering gate); natural and mean-value forms intersected on every box (rock-physics engine)	Eq. (2); cost only
split or stop	widest coordinate (universal branch-and-bound); largest sensitivity (engineering gates: the largest floating-point variation of the model across the box, or the largest $\lvert\partial g/\partial x_i(X)\rvert\, w(X_i)$ after the one-signed coordinates are fixed [24, 75]); stop at a depth of 12 splits per branch (gate), a depth and a box budget (branch-and-bound, by default 9 and 2×10^6), or 6,000 evaluations (rock physics)	Proposition 3; cost only
which certifier composing checks	by the question's form (Table 2) minimum in REFUTED $\prec$ REFUSED $\prec$ CERTIFIED	the theorem of each row Eq. (7), Corollary 1
kind of refusal	split if both witnesses are proved; unresolved otherwise	Definition 3
threshold	closed form when the parameter enters additively; otherwise one-sided bisection over nested boxes (sixteen steps in the gate, ten in the rock-physics engine)	Proposition 4, S2
accepting a battery	all red controls fire	protocol rule R2 (Section 2.4.2)

2.3.3. Evaluation design

The evaluation answers Q3 with a benchmark of known truth and Q1 and Q2 with observational records.

Benchmark with known ground truth. The benchmark is the system's own public register, mutated. Each register row that carries a published finite object—a tensor decomposition, a point configuration, a set of circles or points, a printed constant, a polynomial map, an entropy law, a truth table—is an *original*; from it the benchmark builds *mutants* of three kinds, each with a label known by construction: *false* (a coefficient raised by one, a digit moved, a point displaced, a bar raised one unit past the certificate's own printed end), *true* (an invariance of the claim: a translation, a rotation, a relabelling of coordinates), and *malformed* (a missing exponent, a weight off a probability law, a digit beyond the alphabet). Every label is established by an *oracle* written for the benchmark in the Python standard library—exact fractions, directed decimal rounding, integer convolutions—that shares no code with the deciders it judges; a false mutant counts only when the oracle exhibits the violating element (a *witnessed* false claim), and a mutant the oracle cannot witness is discarded and counted as such. The design, the classes, the kill switch (the campaign stops if fewer than half the rows yield a witnessed false mutant) and the outcome words were pre-registered before the first mutant ran; the note is pinned by digest in every record, and its five dated amendments moved no definition. This is the planted-solution design of benchmarks for other verifiers [76] applied to the system's own record. Every claim is also decided by a baseline (B0): a floating-point checker that evaluates the predicate in double precision at a tolerance $\tau \in \{10^{-6}, 10^{-9}, 10^{-12}\}$, of the kind shipped with the releases audited here [11, 77].

The arithmetic against an independent library. The interval kernel's operations and functions are compared, case by case on seeded inputs, with the exact value or range computed in Arb [23]: the rule is containment, and two planted wrong kernels must be caught. This answers Q2 on sampled inputs; the lemmas of S1 answer it on paper.

Coverage across domains, and the register. Q1 is answered by reporting, for each domain, how many claims could be stated in the form of Definition 2 and with which verdicts they were decided, and by listing what could not be so stated. The register of machine-generated claims [15] is reported as an observational study: how many claims of each source certify, refute or refuse, and what kinds of defect the refutations expose. It measures what independence finds in practice, not a false-certification rate, since the ground truth of its claims is not known independently.

2.4. Software architecture

2.4.1. System overview

The mathematics of Section 2.2 is implemented in four layers (Fig. 1): an arithmetic layer (exact rationals, outward-rounded intervals, function enclosures, compensated sums and derivatives); a certifier layer (the certifiers of Table 2 and the decision loop of S4); a record layer (certificates, witnesses, digests and the register's builder); and a verification layer (the batteries, red controls and second implementations of Section 2.4.2). A *proposer*—a search, a model, a language model or a printed table—lies outside the trust path: its output is read as a claim over a box and decided by a *certifier*. A floating-point *screen* may discard candidates before certification but can never admit one. The verdict, its witness and the inputs pinned by SHA-256 form a *record*.

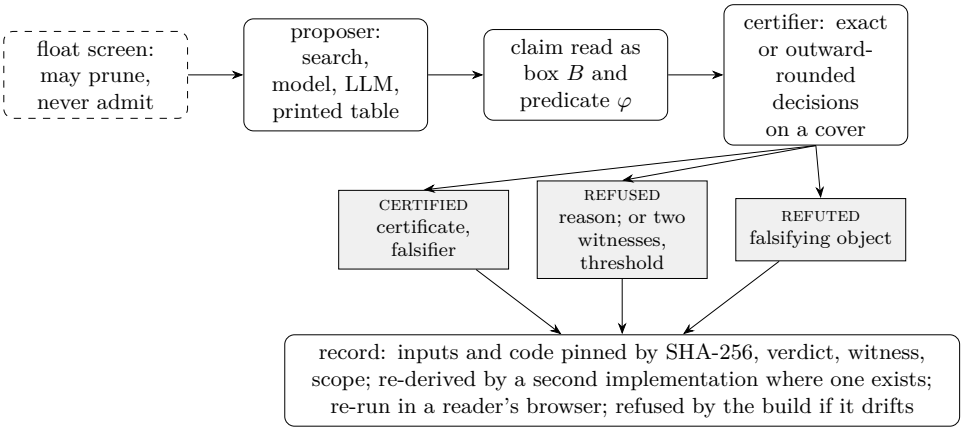


Figure 1: The cert-machine pipeline. The proposer lies outside the trust path; its output is decided by a certifier, and the verdict, its witness and the pinned inputs form a record.

2.4.2. Verification protocol

Proposition 2 makes the verdicts as sound as the code that computes the extensions. The protocol below is how that code is guarded. Each rule is an executable gate—a test battery that exits non-zero, a page builder that will not write its page, a numbers generator that will not write a paper's macros—and Table 4 states which gates are enforced at every level and which are not yet.

R1. Screens prune, never admit. A floating-point screen may discard candidates; only the certifier can record a hit, and the campaign runner plants known answers that must pass every stage before a run may start.

R2. Adversarial controls must fire. A *red control* is a deliberately wrong variant of a test that must be rejected (a "gate" evaluating the

model only at the centre of a box, which the corner rule must catch); a battery whose red controls do not all fire fails. Red controls are mutation testing [78, 79, 80], by which soundness defects have been found in SMT solvers [81, 82]. Of the 166 batteries the test target runs (105 in JavaScript, 61 in Python), 99 state their red controls, 705 in all.

R3. Pin the bytes. Every claim is decided against a byte sequence named by path and SHA-256 and re-hashed at decision time; certifying modules are pinned in a ledger; a page refuses to build when a pinned input changed, when the record was written by another version of its verifier, or when its battery fails.

R4. Independence from the claimant. The claimant's program is never in the trust path; the decision is re-derived from the published statement and bytes, by a system that shares no code with the claimant, and every exact certificate re-checks with no engine present. The exceptions known at the time of writing are listed in Table 4.

R5. Second implementations. Where a verdict is the product, a program written from the specification in another language and arithmetic, sharing no code, re-derives it (the list is in S5). Sharing no code does not make failures independent [83, 84, 85]; agreement is evidence against defects the two do not share, not proof of their absence (Section 4.4).

R6. Records re-run. Published records re-run in the reader's browser by the bytes that wrote them and are compared with the record by digest or field by field; a rerun kit of 31 commands re-derives the register's records (Section 3.1).

R7. Counting rules. Every number in a paper is a macro written from a record by a generator that refuses when the record no longer supports the sentence quoting it; queued rows are not decided rows; refusal kinds are never merged; a printed enclosure is rounded outward, never cut.

Table 4: Enforcement status of the protocol rules at the time of writing.

rule	enforced per battery / page / record	enforced in aggregate
R1 screens	yes	partly: every family of the generation loop decides through one verdict module; instruments and applications keep their own verdict words, checked against the closed list at publication
R2 red controls	on the 99 of 166 batteries that state red controls; the others check without one	yes: the test target exits non-zero on any failing battery and the control page refuses to build
R3 pinned bytes	yes	yes
R4 independence	by design	no: two instruments reuse the system's own code across a producer–checker line (the mean-field-game capacity certifier imports a frozen earlier verifier; the lemniscate certifier shares the interval kernel with the bench it was lifted from)
R5 second implementations	where a second program exists (S5)	not for every certifier (point-in-polygon, cosine minimum, sums of squares)
R6 re-run	yes	re-runs the record, not an independent code path
R7 counting	yes	yes

3. Results

3.1. The benchmark with known ground truth

Design, as pre-registered. The register held 132 rows from 25 records when the pre-registration was pinned on 2026-10-21. Of these, 119 rows belong to a family whose decider the benchmark can call on a modified object; 97 of them carry a published finite object and were mutated, and 22 were not, each with its reason recorded: rows decided by reading a finite object the claimant did not publish in a mutable form (the counterexample rows), and rows whose object is a bound certified from a literature argument rather than from bytes. The mutation classes of Section 2.3.3 produced 572 mutants: 322 false, 153 true and 97 malformed. The oracle witnessed 295 of the false ones; the other 27, all perturbations of printed coordinates by 10^{-12} in the point-set family, moved the object below the precision its claim is printed at and were discarded, as pre-registered. The kill switch was not tripped: 94 of the 119 rows yielded a witnessed false mutant, against a bar of 60.

The verdict–truth matrix. Table 5 is the result. Of the 295 witnessed false claims, 271 were refuted, 6 were refused (a Gaussian coefficient bound too

Table 5: The benchmark by family (`certs/selfaudit-mutations.json`): rows of the register in the family, rows with a mutable object, mutants by label (false/true/malformed), and the outcomes that would be errors or abstentions. "Not as required" counts the mutants whose decision word differs from the pre-registered required word without being an error: a false claim answered REPAIRED (the printed bound refuted and the certificate's own value certified beside it), one answered UNWITNESSED, and the one malformed term the partial path of a polynomial-map decider read silently (Section 4.5).

family	rows	objects	mutants	F/T/M	false cert.	false ref.	refused (F)	discarded	not as req.
matrix-multiplication tensors	45	45	315	180/90/45	0	0	3	0	0
kissing configurations	11	10	69	39/20/10	0	0	0	0	0
point sets and shapes (EinsteinArena)	28	28	136	80/28/28	0	0	3	27	18
a printed constant (Erdős 852)	1	1	5	2/2/1	0	0	0	0	0
polynomial maps and fields	9	9	35	17/9/9	0	0	0	0	1
counterexamples (finite objects by reading)	14	0	0	0/0/0	0	0	0	0	0
entropy certificates and a truth table	4	4	12	4/4/4	0	0	0	0	0
Turán-type bounds	2	0	0	0/0/0	0	0	0	0	0
HorizonMath rows	3	0	0	0/0/0	0	0	0	0	0
the GNNW constant	1	0	0	0/0/0	0	0	0	0	0
a sum–product bound	1	0	0	0/0/0	0	0	0	0	0
all	119	97	572	322/153/97	0	0	6	27	17+1+1

large for exact double summation in three tensor rows; three flat-polynomial rows whose coefficients, perturbed off the integers, lie outside the decider's schema), 17 were answered REPAIRED—the register's composition of a refutation of the printed bound with a certification of the certificate's own value, which is a refutation of the claim as printed—and 1 was answered UNWITNESSED, a refusal under the ledger's own word. Of the 153 true mutants, 153 were certified, and so were all 97 originals. Of the 97 malformed objects, 96 were refused and 1 was decided: the structural zeros of Eq. (8) hold for false certifications and false refutations and fail once for a malformed object, reported below as an open finding. Table 6 is the verdict–truth matrix over the 545 witnessed claims. Its informative answers number 538 (every verdict but the 6 unresolved refusals by resources and the 1 by the decider's own word), a coverage $C = 98.7\%$; the selective risk from false certifications and false refutations is $R = 0$, and counting the one malformed object decided it is $R = 0.19\%$. The coverage on witnessed false claims is 271 refuted and 17 repaired of 295. With zero false certifications on 295 witnessed false claims the one-sided Clopper–Pearson bound at $\beta = 0.05$ is $\theta_U = 1.01\%$, and with zero false refutations on 250 true claims (mutants and originals) it is 1.19%; both describe this benchmark and its classes, not other claim sets (Section 2.3.1).

Two defects found by the benchmark's rule. The campaign's first run, at commit d961cda2 on 2026-10-21, showed 28 false certifications and 49 malformed objects decided, and both were defects of the system. The first: the ledger of point-set configurations composed its word from whether the

Table 6: The verdict–truth matrix over the witnessed claims of the benchmark (`certs/selfaudit-mutations.json`); the 97 unmutated originals, all CERTIFIED, are not in it. REPAIRED is a refutation of the claim as printed; the one PARTIAL is the open finding of Section 4.5.

verdict	true (153)	false, witnessed (295)	malformed (97)
CERTIFIED	153	0	0
REFUTED (of which REPAIRED)	0	271 + 17	0
REFUSED, unresolved	0	6 + 1	—
REFUSED, by the shape check	—	—	96
another word (PARTIAL)	0	0	1

published bytes witnessed a value, while the agreement of the printed digits with that value was a field beside the word, gated only by the page build; a printed value moved one unit in its last digit past the bytes therefore stayed CERTIFIED. The fix composes the word in one module—a witnessed object whose printed value the exact value does not reproduce is MISPRINTED, the register's REFUTED with the kind *not from the published data*—and the ledger was re-derived under it with no row changing word. The second: the tensor and kissing deciders answered a malformed object (a tensor with the wrong number of slices, a configuration that is not n vectors in $\mathbb{R}^d$) with the word REFUTED, the identity having failed; the fix makes every shape check answer REFUSED with its reason. Both fixes are pinned in the campaign's record with the commit each was found at, the campaign was re-run whole after them, and Table 5 is that re-run. The pre-registration's rule—a defect found by the campaign is fixed on the record and the campaign re-run, never the definition moved—was followed; each of the five amendments is dated and lists what it does not change.

The conventional check, on the same claims. The baseline B0 evaluates each predicate in double precision at a tolerance. At $\tau = 10^{-6}$ it accepts 27.8% of the witnessed false claims (82 of 295), at 10^{-9} 11.2% (33) and at 10^{-12} 2.4% (7). The tolerance cuts both ways: at the same three settings it confirms only 94, 81 and 72 of the 97 true originals, and 149, 136 and 127 of the 153 true mutants, because four of the nine polynomial maps are decided by no tolerance at all: their determinants are exact integers that double precision does not hold. On the risk–coverage plane of Section 2.3.1 the checker decides every claim it can evaluate in double precision (the four polynomial maps it cannot are its abstentions) and accepts between 2.4% and 27.8% of the witnessed false claims at the three tolerances; the decider has $R = 0$ on those claims and $C = 98.7\%$.

Concordance, re-runs and cost. Where a register row is decided by two or more programs—the deciders under test, the benchmark's oracles, the rerun kit's standard-library verifiers, the second implementations of rule R5—the

concordance record compares their answers field by field: 91 comparisons on 56 rows—39 on the enclosure or the violating count beside the word, 49 on a re-derivation's exit and three on the exact object alone—with no disagreement. The rerun kit, run whole on 2026-10-21, moved none of the 132 rows; the campaign re-derived the register's records in 124 runs under two profiles and found every record identical to its pin outside declared time stamps (23 records and 125 rows without a wall-clock cap; 18 records and 113 rows under a 60-second cap, the other 5 refused for resources). The 416 kernel replays of the mathematics release ran on hosted runners and are carried as recorded, never merged into the laptop's table: 398 CERTIFIED under two kernels, 18 REFUSED for resources, wall-clock from 18 to 12,453 seconds (median 587) and peak memory up to 13.4 GiB. The mutation campaign took 17 minutes on the laptop (Apple M2, 8 cores, 16 GB, Node 24.14.1); the re-derivation of the records took 26 minutes of wall-clock under the extended profile and 9 under the portable one, with a peak resident memory of 0.66 GiB. Per record (S5, Table S2), the median re-derivation takes 3.4 s, 18 of the 23 records finish every command within a minute, and the longest, the likelihood fits, takes 536 s. Nothing was rented.

What the benchmark does not cover. Every claim in it is universal or exact, so the epistemic reading, the split refusal and the undetermined label are not measured; the engineering gate's planted faults (7 faulty proposals, 6 refuted and 1 refused [2]) are the only ground truth of that kind and are targeted, not sampled. The mutation classes were chosen by the first author, so the benchmark bounds the defects it can find by the classes it plants. No comparison with another exact decider was run: B0 is the only baseline reported, and a comparison with δ-complete or interval-library deciders on the claim classes they can express is future work.

3.2. The kernel against an independent library

The interval kernel was run on 1,012,121 seeded cases across 24 operations—up to 50,000 each, drawn from full-precision doubles, subnormals, arguments near overflow and underflow, and boxes that straddle a function's critical points—and every enclosure it returned was compared with the exact result, or for a box the exact range, computed in Arb [23] at 256 bits and more (Table 7). Every one of the 1,009,497 enclosures it returned contained the exact value, and it refused the other 2,624; none of the 12 malformed inputs (a NaN, an inverted interval, division through zero, a non-integer or overflowing power, a logarithm at or below zero) was decided; and the comparison caught two planted wrong kernels, a round-to-nearest addition in 19,846 of 50,000 cases and the platform's exponential at the ends in 50,000 of 50,000. The audit covers the special functions the likelihood certifier uses—the gamma function by Spouge's

formula, the Bessel function, the inverses of the normal and incomplete-gamma distributions, each judged against a bracket bisected in Arb on certain signs—and the compensated sum against the exact rational sum of its terms. The widths tell where the enclosures are loose: the gamma function's tenfold Spouge margin and the Bessel series near the zeros of J_ν are contained by many ulps, which costs tightness, never soundness.

operation	cases	contained	violations	refused	median ulps
$x + y$	50,000	50,000	0	0	2
$x - y$	50,000	50,000	0	0	2
$x \times y$	50,000	50,000	0	0	2
x/y	48,529	48,529	0	0	2
x^2	50,000	50,000	0	0	2
$\lvert x \rvert$	50,000	50,000	0	0	0
x^k	50,000	50,000	0	0	9
exp	48,289	46,799	0	1,490	120
log	49,838	49,838	0	0	5
sin	50,000	50,000	0	0	365
cos	50,000	50,000	0	0	328
tanh	50,000	50,000	0	0	0.5
the platform's cos, padded	50,000	50,000	0	0	4,816
the platform's sin, padded	50,000	50,000	0	0	4,820
$\ln \Gamma$	50,000	50,000	0	0	46
ψ	50,000	50,000	0	0	36
ψ'	50,000	50,000	0	0	22
$P(a, z)$	50,000	50,000	0	0	7,061
Φ	49,465	49,465	0	0	117
Γ (Spouge)	50,000	48,866	0	1,134	21,638,632
J_ν	5,000	5,000	0	0	1,098,797
Φ^{-1}	5,000	5,000	0	0	240
P^{-1}	5,000	5,000	0	0	5,934
Sum2, n terms	1,000	1,000	0	0	—

Table 7: The interval kernel against Arb (`certs/kernel-arb-audit.json`): for each operation, the cases run, the enclosures that contain the exact value computed in Arb, the violations, the in-domain refusals, and the median width of a point input's enclosure in units in the last place of the larger end of the result (a median below one ulp, as for tanh, comes from results just below 1; no median for Sum2, whose inputs are streams). The widths of the padded platform functions are those of the older helper that Section 4.5 names.

The standard's test suite. The same kernel was run on ITF1788 (`https://github.com/oheim/ITF1788`, commit b6ee1e24), the IEEE 1788-2015 test framework discussed by Benet, Ferranti and Revol [38]: 19 test libraries holding 9,542 hand-written cases from libieeep1788, FI_LIB, MPFI, C-XSC and the standard's examples, each giving the tightest interval an operation must return, read here by a parser that executes nothing of the suite. The 1,891 cases of operations the kernel implements with the standard's meaning and without decorations or the empty set were run: no result failed to

contain its expected interval; 466 were the tightest interval, 974 valid and wider (the one-ulp outward step), and 449 inputs were refused by design (a divisor containing zero, $0 \cdot \infty$, an infinite endpoint, the exponential's reduction range). The suite found one defect, in a predicate rather than an enclosure: the membership test answers true for $\pm\infty$ against an interval with that infinite end (two cases; Section 4.5). The two planted faults were caught (round-to-nearest addition on 19 of 98 addition cases, the platform's exponential at the ends on 32 of 56). The other 7,651 cases—decorations, the empty set, reverse and set operations, and operations the kernel lacks—were counted and not run; the measurement is of soundness on the kernel's own operations, not a conformance claim (`certs/kernel-itf1788.json`).

Refusals by cause. The refusals are enumerated by cause. Of the 416 Comparator challenges of the mathematics release, 18 are refused for a stack, memory or time limit that more resources might clear. Of the 1,650 cosine-minimum decisions of the generation loop, 71 are refused because the enclosure straddles the bar, which a finer cover or a higher-order form might separate. The kernel's 2,624 refusals near underflow and overflow are a design choice: the enclosure $[0, 2^{-1074}]$ would be valid at underflow, and the kernel does not return it. These remain refused, with their cause recorded, pending a stronger method.

3.3. The register, and its three largest exercises

The public register holds 132 claims from 59 claimants, decided from 25 records between 2026-08-25 and 2026-10-05 and frozen on 2026-10-27; decisions made after the freeze are not register rows. Of its rows 131 are decided and 1 is queued, not counted: 107 CERTIFIED, 9 PARTIAL, 2 REFUTED, 5 MIXED, 6 REPAIRED and 2 NEEDS DATA. 24 rows carry a kind of defect other than none; the most frequent is *narrower scope* (8 rows), a formal statement or a check that covers less than the claim beside it. Of the 107 decided rows whose claimant is a machine or a machine-assisted pipeline, 17 carry a defect; of the other 24, 7. The register measures what independence finds, not an error rate: no row's truth is known independently of the decision. Three exercises, begun inside the register and continued after its freeze, are told below from their own records, each with a public report in the archive [1].

A configuration of points. An autonomous discovery system published new lower bounds for the kissing number in 19 dimensions (25, 27, 32–39, 43, 45 and 49–55), each a configuration of integer or algebraic vectors, their sizes proved by moment identities and the angular condition by structure, with the release's own files recording that the pairwise inner products were not checked [11]. Two configurations were published as exact objects and decided at once; the other seventeen were NEEDS DATA with the missing

files named, and the claimant published them within days [86]. Every pair was then decided exactly: 9.75×10^{15} pairs, 6.72×10^{12} at exactly sixty degrees and none closer, in three arithmetics—single-precision BLAS on the laptop, exact algebraic integers for the two closed-form constructions, and half-precision tensor cores on one rented GPU for the seven largest (6.8 GPU-hours, $8.15 billed)—the floating-point paths exact under a Cauchy–Schwarz bound asserted on the data before any path runs and, for the GPU, under the stated assumption on tensor-core accumulation (Section 4.4). All 19 bounds hold; the claimant linked the check from its release. The record (`certs/kissing-ledger.json`) and its report give the materials, the calibration, the second implementation and the limit of the GPU argument.

A curated record. An optimization-constants registry marks with an asterisk a bound whose verification is minimal and states what removes the mark [77, 87]. Three asterisked bounds rest on finite certificates—two entropy laws for the sums–differences constants and a truth table for the Fourier entropy–influence constant—and were decided twice from the registry's bytes by programs that share no code with each other or with the claimants, both written by AI agents under the first author's direction; all three are CERTIFIED with their ratios enclosed. The registry merged the replay and removed the marks, and its pages cite the replay; over three rounds, the last on 2026-10-26, ten marks have been removed on replays made here, and on 2026-10-26 the registry corrected one of its rows on the check. The registry's open queue offered 10 further bounds: 7 are CERTIFIED by a computation made here, one on an earlier two-kernel record, one was checked by reading alone and is counted as a reading, and 1 is REFUSED—a package that certifies its own assembly and not the analysis it assembles, with the three inputs that would decide it named. One bound was printed on the wrong side of its own certificate, $C_{1d} \le 1.463071582607559$ against the certificate's exact $1.46307158260755932746\ldots$; the record holds the printed bound REFUTED and the repaired one CERTIFIED. The sums–differences certificate came with a checker of its own, which encloses the ratio at a hundred digits and compares in double precision: given 4 claims here, it printed OK on all of them, 2 of which are false. On 2026-10-26 the registry's contributing rule was extended to require a second party for the removal of a mark, by one of four routes, one of them an independent confirmation on the thread; on 2026-10-27 two graphon bounds submitted by another contributor were decided here from their pinned certificates by two implementations sharing no code, with 5 controls, and both confirmations were posted as that second party.

A formal release. A mathematics release of 719 machine-written manuscripts came with 416 Lean challenge statements as its formal spine, and with a Navier–Stokes repository proved against a statement

written months earlier [13, 12]. Every challenge was replayed under Lean's kernel and under a second kernel written independently of it: 398 accepted by both, 18 REFUSED for resources, none rejected. The Navier–Stokes development built clean at its pinned commit with the standard axioms, and its formal statement, read hypothesis by hypothesis against the paper's theorem, lacks the energy clause of that theorem; the record is PARTIAL at 8937a8f4 with the clause named in its scope, and the next upstream commit, f9e8bc5b of 2026-09-10, proves it. Of the 242 manuscript families whose statements were read against their theorems, 127 match, 34 state less than, or other than, what they claim, 38 are narrower and declared so, 39 are not linked to a theorem, and 4 are supporting lemmas; the readings were made by AI agents and are not verdicts, and their recall was measured once, on 12 defects planted by the first author in short statements across 24 packets with 12 unaltered controls, 12 flagged. The records (`certs/openai-math-kernel.json` and the readings beside it) and the report give the kernel lane, the readings and the 14 gaps the audit pinned.

3.4. Coverage across domains

Q1 asks whether one grammar covers claims from mathematics, from engineering and from metocean statistics. Table 8 lists, for each domain, the claims stated in the form of Definition 2 and their verdicts, each read from the record that holds it. In mathematics the register's rows are the claims. In engineering the flow-line gate decides the demonstrator's receipts over a declared box of six inputs under the epistemic reading—the lead proposal split, with the pump pressure at which the rule is proved (204.0 bar) and refuted (222.3 bar) and the roughness measurement that would prove it (27.000 mm) reported as thresholds, its fixed version proved—and 7 faulty proposals planted beside it, 6 refuted and 1 refused [2]; the rock-physics engine decides detectability over 360 declared petro-elastic cells under the same reading, with the counts in its study [35]. In metocean statistics the likelihood certifier decided 46,602 fits of six families at 2,589 open-sea cells of a public hindcast: 40,778 CERTIFIED (87.5%), 5,824 REFUSED, of which 5,751 are the generalized gamma proved at its lognormal limit with the proof; at the benchmark's buoys 353 of 402 fits are CERTIFIED and every ranking of families is decided or refused by name [3]. What could not be stated in the grammar is listed with the register: a claim about an object the claimant did not publish is NEEDS DATA, and a statement decided by reading alone—a formal statement against its theorem, an inventory of a package—is recorded as a reading and never as a verdict.

Table 8: Claims decided by one grammar, by domain, each count read from the record that holds it. The engineering and metocean rows are the headline counts of their studies [3, 2]; the register's composite words are grouped.

domain, instrument	claims	CERTIFIED	REFUTED	other	other, by word
mathematics, the register	132 rows, 59 claimants	107	2	9+5+6+2+1	partial, mixed, repaired, needs data, queued
engineering, the flow-line gate	7 planted faults; one lead proposal	0	6	1+1	refused (split, with thresholds)
metocean, the likelihood certifier	46,602 atlas fits	40,778	0	5,824	refused, 5,751 at the lognormal limit with the proof
metocean, the benchmark's buoys	402 fits	353	0	402 less 353	refused by name

4. Discussion

4.1. What the results show

Measured soundness. Proposition 2 predicts zero false verdicts, and on 572 claims of known truth the implementation produced no false certification and no false refutation, after two defects that the benchmark's own rule exposed were fixed on the record. Neither defect was in the arithmetic: one was a ledger that said CERTIFIED about the bytes while the printed digits beside them disagreed, the other a verdict word given to an object that was not a claim at all. Both are gaps between what was checked and what the word said, and both were found by planting claims against the system's own register, which is what the protocol asks of claimants. Writing the lemmas of S1 against the code found eight more places where the implementation was weaker than the stated lemma, and an outside rerun and the sweep it started found four programs that printed an upper end cut toward zero rather than rounded outward (Section 4.5); each lay below the enclosures' widths, none moved a verdict, and each was unproved until written down.

Defects found in the register. No bound of the three exercises was found false beyond its last printed digit and no theorem was false. One registry bound is printed on the wrong side of its own certificate; a pairwise condition was proved by structure and never checked pair by pair; a certificate's checker compares in double precision what it encloses at a hundred digits; a package certifies its own assembly and not the analysis it assembles; a clause of a theorem is absent from the formal statement. Each was closed, or named, by a finite computation or a recorded reading. A refusal carries its threshold or its missing input: seventeen kissing rows were NEEDS DATA until the points were released, each naming the files that would decide it; the gate's split refusal carries the pump pressure at which it would become a proof; the package is REFUSED with its three missing inputs named. The cost of exactness is small: the objects are integers, rationals and bounds, and the cost of the one expensive decision—9.75×10^{15} pairs—scales with the number of pairs, not with the exactness of the arithmetic.

Outcomes following the decisions. Each exercise ends outside the system. The kissing claimant published its points in the form the check needed and linked the decision from its release. The registry's maintainer, who had written down beforehand what removes the mark of an unverified bound, removed ten marks on the replays, corrected a row on the check, cites the replays, and on 2026-10-26 made a second party a requirement for every removal; the first confirmations under that rule were given here, for another contributor's bounds. The release's next commit, made the day after the audit pinned the repository, proves the clause the reading found missing. We draw no causal conclusion from these cases. They show what the grammar makes possible: a CERTIFIED with its certificate can be cited, a REFUSED that names its files can be answered, and a scope sentence can be closed.

4.2. Independence, by level of evidence

We distinguish five levels of evidence of independence (Table 9) and place every result of this paper on them. Every computed result reaches the first level; most reach the second; the third is reached for parts of claims only—the kernels' claim, the arithmetic on sampled inputs, and the rows outside parties re-derived with their own code—and none reaches the fourth or the fifth. Outside parties have re-derived 7 records of the repository (3 parties, 2026-09-30 to 2026-10-24, each with the same verdicts; `corpus/external-reruns.json`): 5 of them on rows of this paper's exercises, by 2 parties, of which 3 with their own code (twelve register rows, the registry's C_{31} and C_{84b} rows: level 3 for the parts re-derived) and 2 with the tools that write the records (the three ledgers and the C_{42} record: level 1 by the table's definition); and 2 outside it, the $\lambda(4)$ audit [88] and the Ramsey graphs. One of the tool re-runs read the C_{42} decider and found three defects in what it printed, none moving a verdict, fixed and recorded in the benchmark's amendments. The registry's merges are not on the table at all: its maintainer applied his rule to the replays and did not re-derive them, so they are outcomes, not checks.

4.3. Relation to existing tools

Table 10 places the system beside the tools that share its ingredients, by the properties that Section 2.2.2 and Section 2.4.2 define. Interval libraries [22, 23, 38] and interval global optimisation [24, 41] compute the enclosures; δ-complete solvers [30, 89] decide formulas over the reals up to a user-chosen perturbation and can export checkable traces [90]; proof assistants check a certificate against a formal statement inside a trusted kernel, with independent kernels that replay the check [31, 32, 33, 91], but whether the statement is the theorem intended is not checked [16, 92]; certifying algorithms and proof-checking solvers return a witness a small checker verifies [51, 53, 54]; reproducibility initiatives re-run or re-implement published computations [93, 94]; and certificate architectures for language-model

level	evidence	where it holds in this paper	what it cannot catch
1	the same code, rerun from pinned inputs	every record re-read against its pins at build; the register re-derived by the rerun kit on 2026-10-21 (0 rows moved) and by the self-audit under two profiles (23 records identical outside stamps); 2 outside re-runs of the ledgers' own tools	any defect in the code itself
2	a second implementation by the same author, from the same specification	the benchmark's six oracles, the kissing samples, the entropy and truth-table certificates, the step functions, the trace-constant and Zhang–Zagier certificates, the graphon densities, δ_3, the Hankel cross-checks, three engineering decisions; written by AI agents from one brief	a misreading the two share
3	an implementation by another party	the Lean kernel and a second kernel for the kernels' claim (Section 3.3); Arb for the interval kernel's arithmetic on 24 operations and for the entropy deciders' arithmetic, on sampled inputs (Section 3.2); 3 outside reruns with their own code on the exercises' rows (twelve register rows, C_{31}, C_{84b}), same verdicts; outside this paper, the $\lambda(4)$ audit and the Ramsey graphs	a defect outside the part re-derived
4	review of the specification and the scope by a specialist	none: the readings were made by AI agents and no specialist has confirmed them	—
5	a formal proof of the metatheory	none: Proposition 2 and the lemmas are paper proofs; the exact-class certificates are in a form a proof assistant could check	—

Table 9: Five levels of evidence, and where each result of this paper stands on them; the levels are reported separately.

pipelines treat abstention as an outcome [95, 62]. Each enforces one or two of the properties; none, to our knowledge, is organised around other parties' published numbers, and none reads a printed digit string as the box it denotes or distinguishes the universal from the epistemic reading. The combination, enforced by code that refuses, is what is claimed; the properties the system itself meets only in part are marked as such, with the reason in Table 4 and Section 4.5.

Table 10: Properties of the decision, by family of tool: whether the tool decides another party's published claim from its statement and bytes; reads a printed value as the box of Eq. (4); separates the universal from the epistemic reading; publishes a failure to decide as a result with its reason (and a threshold where a parameter is declared); returns a witness that a program other than the tool can check; and enforces, by code, that the producer's program is not in the trust path. "Partly" is explained in the text and, for cert-machine, in Table 4; for cert-machine the witness column reads "yes" for the exact class and "partly" for the interval class, whose certificate records the hull and the counts of its cover, not the cover.

tool family	other party's claim	printed box	two readings	refusal as result	witness	producer out
interval libraries, ball arithmetic [22, 23]	no	no	no	partly	no	n.a.
interval global optimisation [24, 41]	no	no	no	partly	cover	n.a.
δ-complete solvers [30, 89, 90]	no	no	partly	no	trace	n.a.
certificates in a proof assistant [31, 32, 33]	partly	no	no	no	yes	partly
certifying algorithms, proof-checking solvers [51, 53, 54]	n.a.	no	no	no	yes	n.a.
reproducibility re-runs [93, 94]	yes	no	no	partly	no	partly
certificates for LLM pipelines [95, 62]	partly	no	no	yes	partly	no
cert-machine	yes	yes	yes	yes	yes / partly	partly

4.4. Threats to validity

- *Selection.* Every claim was chosen by the first author: the register records every row as self-initiated, and the exercises were picked because they had finite objects to decide. The results describe these claims, not machine-generated mathematics at large.
- *The benchmark's classes.* The mutation classes were designed by the first author, so the benchmark can find only the defects its classes reach; the Clopper–Pearson bounds describe this benchmark and do

not transfer. Every claim in it is universal or exact, so the epistemic grammar is measured only by the gate's targeted faults.

- *The author in the outcomes.* The outcomes of Section 3.3—the points published, the marks removed—followed requests and pull requests the first author made. They are reported as what happened, and no causal claim rests on them.
- *Correlated implementations.* The deciders, their second implementations and the benchmark's oracles were written by AI agents under the first author's direction, from shared briefs. N-version programs written that way can fail together [83, 84, 85], and level 2 of Table 9 is counted apart for that reason.
- *The arithmetic's evidence.* The comparison with Arb covers sampled inputs, not every input; the lemmas cover the functions, not the code that implements them; the standard's test suite was run only on the kernel's own operations, without decorations or the empty set, so it measures soundness there and not conformance.
- *The readings.* The statement readings and the Navier–Stokes reading were made by AI agents and checked only by second readings made the same way; their recall was measured once, on short statements under an alert brief. No human specialist has confirmed them.
- *The GPU argument.* The exactness of the seven largest kissing decisions rests on the documented single-precision accumulation of tensor cores [96, 97, 98], characterised for earlier devices than the one rented, on the bound asserted before the run, and on a calibration that reproduced the laptop's exact contact counts on two configurations; the calibration is a check, not a proof.

4.5. Scope and limits

Not claimed. Interval arithmetic, the Krawczyk operator, floating-point filters, sums of squares and Sturm sequences are cited, not claimed. The validity of a declared model is not claimed: a PROVED engineering number (the demonstrator's word for CERTIFIED) holds for the declared model over the declared envelope, not for the sea or the reservoir. A kissing bound "decided here" is a statement about one published configuration, not about the record in that dimension. A certified maximum is the maximum in its box; a global statement over a family would need a branch-and-bound over the family's whole parameter space, which the system does not run. Nothing is certified in the regulatory sense; the word is used in the sense of Definition 3 only. Nothing is a probability.

Three results of the register rest in part on readings rather than computations—the clause found absent from the Navier–Stokes formal conclusions, the statements of the release found short of their claims, and

the inventory of the record package—and are reported as what the readings found, with their recall as measured (Section 4.4).

Soundness rests on code. Proposition 2 is a theorem about interval extensions; whether a program computes one is a fact about the program. The defences—second implementations, red controls, pinned bytes, calibrations, the planted-claim benchmark, the comparison with Arb, outside reruns—reduce that risk and do not remove it, and what they found is on the record. A read of the code on 2026-10-09 found that the generation loop's closed-form hunt had counted 54.6 million float comparisons as refutations; every refutation is now a proved disjointness, with the correction published beside the old numbers. The benchmark's two findings are in Section 3.1. Writing the lemmas of S1 found, and the build of this version closed, with every record that pins the changed modules re-derived: a square root of π taken from a thin box of the double nearest π, which does not contain π; an error-function tail short of the alternating-series bound by its own factor $2/\sqrt{\pi}$; a gamma-function margin computed in the platform's floating point; four transcendental tails evaluated in round-to-nearest; the lower end of $\ln\Gamma$ across its minimum as a hard-coded double; the flip thresholds rounded to nearest before the decimal rounding; the gate's witnesses one ulp outside the declared box; and the kissing runner recording an exactness bound it did not assert. An outside rerun of the C_{42} decider and the sweep it started (2026-10-24 to 2026-10-27) found four programs—three deciders and one of the benchmark's oracles—whose *printed* upper end was cut after k decimals rather than rounded up, so the digits shown lay below the enclosure they reported; every decision was made on the full enclosure and none moved, and every printed end is now rounded outward. Each of these lay below the widths in play; each was a place where the implementation was weaker than the stated lemma. What remains open:

- the older helper that encloses the platform's cosine by a derived pad, contained in every case of Table 7 but wide, is still called from six files, among them the radii-polynomial certifier, whose records are re-run when it is retired;
- the interval kernel's membership test answers true for $\pm\infty$ against an interval with that infinite end (Section 3.2); every call in the repository passes a finite point (a constant or a computed double), and the one-line fix moves the pins of four records, which are re-derived when it is made;
- the partial path of one polynomial-map decider reads a term with a missing exponent silently and answers PARTIAL where its full path refuses (the one malformed object decided in Table 5);
- one quadrature routine, rigorous on dyadic cells of $[0, 1]$, is called on another interval by one instrument; one battery labels some positive

assertions as red controls; in the engineering demonstrator a PROVED consistency check means that the claim meets an outer enclosure;

- two of the system's own instruments reuse its code across a producer–checker line (Table 4), and the same-author second implementations are not independent re-derivations (Table 9);
- only the exact-class certificates are in a form a program other than this system's can check; an interval certificate records the hull and the counts of its cover, not the cover.

5. Conclusions and recommendations

cert-machine decides the numbers and claims that machines propose with three verdicts, each carrying the object that justifies it. The interval and exact verdicts are sound under the inclusion property, which the arithmetic is built to secure and whose function enclosures carry a stated lemma each; the certifiers reduce the questions that arrive—a zero, a maximum, a statement for every input, a sign, a nonnegative polynomial, a minimum, a formal proof—to finitely many exact or outward-rounded decisions; the protocol is enforced by executable gates; and the record is public and re-runs. Measured against 572 claims of known truth, the system produced no false certification and no false refutation and refused 6 for resources; measured against an independent library on 1,012,121 cases, every enclosure held; applied to 132 machine-generated claims, it found defects of reporting rather than of mathematics, after which the claimants published their points, the registry removed its marks and corrected a row, and the next upstream commit of the release proved the missing clause—outcomes reported without a causal claim. The three questions are answered as far as the record allows: one grammar covers the claims of Table 8 in the three domains (Q1); the hypothesis holds on the inputs tested and is proved on paper for the functions of Table A.1 (Q2); and on claims whose truth is known the system certifies what is true, refutes what is false and refuses a stated fraction by resources, the benchmark taking 17 minutes on a laptop and the exercises the costs of Section 3.3 (Q3).

Recommendations. Three practices would make such claims decidable from their publication. (i) Producers publish the object, not only its description—the points of a configuration in one integer form, the witness a theorem rests on, the analytic inputs a certificate freezes, the data a fit was made from—with a digest beside each file, state the scope of every check in the release itself, and say where the theorem is stated when a formal statement is narrower than it. (ii) Maintainers of the tables and records of a field distinguish a bound decided from outside from one only claimed, and write down what removes the distinction. (iii) Builders of checkers print every

enclosure outward and plant claims of known truth against their own record, with an oracle that shares no code with the decider.

Declaration of competing interest

The authors declare no competing financial or personal interests. The claimants named in Section 3.3, the maintainer and contributors of the optimization-constants registry, and the maintainers of the benchmarks and tables named here have not been consulted on this draft; the exchanges with the kissing-number group and with the registry took place on the public threads cited.

CRediT authorship contribution statement

Carlos Toledo: conceptualization, methodology, software, validation, investigation, data curation, writing—original draft, writing—review and editing. Pedro Veras Guimarães: methodology, formal analysis, writing—review and editing. The roles are to be confirmed by the co-authors.

Data availability

Every record, certifier, figure source and table source is public in the cert-machine repository (`https://github.com/carlostoledo1891/cert-machine`, commit 1eb3cd33c866; archived as [1]): the register `certs/claims-ledger.json`; the benchmark's pre-registration `notes/selfaudit-preregistration-2026-10-20.md`, its four records `certs/selfaudit-*.json` and its oracles under `instruments/selfaudit/`; the kernel audit `certs/kernel-arb-audit.json`; the outside reruns `corpus/external-reruns.json`; the records of the three exercises, `certs/kissing-ledger.json`, `certs/sumdiff-ledger.json`, `certs/fei-ledger.json`, `certs/optq-ledger.json`, `certs/graphon-replay.json` and `certs/openai-math-kernel.json`, each with its report under `reports/`. Every constant in this paper is interpolated from one of these records at that commit, so that the document cannot quote a number the machine did not certify, and every bibliography entry is resolved at its source by the build's reference check. The headline results re-derive with the commands of the reproduction table (S5). The claimants' bytes are cited at their sources and pinned by SHA-256 digest in the records; they remain under their owners' licences. The code is MIT-licensed.

Declaration of generative AI and AI-assisted technologies

During the preparation of this work the author used large language models (Claude, Anthropic) to draft code, exposition and this manuscript in a

certified-verification setting: constructions and prose were proposed with the help of the models, and every quantitative claim is decided by outward-rounded interval arithmetic or exact rational arithmetic, with adversarial controls that must fire for a record to be accepted. After using the tool the author reviewed and edited the content and takes full responsibility for the content of the publication. The records, not the models, are the evidence offered; they are public and re-runnable. For this paper specifically: the deciders, the certifiers' code, the benchmark's oracles and the readings named in Section 3.3 were produced by AI agents (Claude) under written briefs and consolidated into the public records; no verdict here rests on a reading alone, and no human specialist has confirmed the readings (Section 4.5). Referee-style readings of the previous versions by an AI agent, and the co-author's restructure of the methodology with its line-by-line code checks, were applied in this version.

Funding

This work received no external funding. The rented compute named in Section 3.3 ($8.15 for the seven largest kissing configurations; the challenge re-runs within a pre-registered cap) was paid by the first author.

Appendix A. The arithmetic in detail

Proof of Proposition 1. For these operations the extremes of $a \circ b$ over $A \times B$ are attained at endpoint combinations (monotonicity in each argument on each sign region, with $0 \notin B$ for division). The exact extreme lies within half the spacing of the doubles on its side of the rounded value, and the adjacent double on that side is a whole spacing away—also when the rounded value is a power of two, where the spacing below is half the spacing above, and in the subnormal range—so it lies on the correct side of $\mathrm{down}(y_-)$ and $\mathrm{up}(y_+)$. □

Table A.1 lists every elementary and special function the certifiers call, with the lemma of S1 that establishes Assumption 1 for it. Throughout, a series is evaluated by the natural extension on the enclosed argument, the tail bound is evaluated in interval arithmetic and its upper end $\overline{R}$ taken, and $[-\overline{R}, \overline{R}]$ (or $[0, \overline{R}]$) is added by an outward-rounded addition, so the bound is never absorbed into the widening of Proposition 1. The enclosures of π and $\ln 2$ are exact rationals (Machin's formula; $\mathrm{atanh}(1/3)$) converted outward, and every constant below ($\sqrt{\pi}$, $\sqrt{2\pi}$, $\pi/2$) is derived from them.

Table A.1: Elementary and special functions, how each enclosure is obtained, the lemma of the supplement that proves it, and whether the function is among the 24 operations compared with Arb (Section 3.2; evidence on sampled inputs, not a proof).

function	method	proof	Arb
exp	reduction $x = k \ln 2 + r$, $\lvert r\rvert < 1$; 24 Taylor terms; geometric tail $\rho^{25}(25!)^{-1}(1-\rho/26)^{-1}$	Lemma S1.1	yes
log	$x = \mathrm{m}\, 2^e$; series of atanh in $v = (\mathrm{m}-1)/(\mathrm{m}+1) \in [0, 1/3)$, 40 terms, one-sided tail	Lemma S1.2	yes
sin, cos	reduction by an enclosure of $\pi/2$; 16 (sin) and 17 (cos) Taylor terms, Lagrange tail; a box's range forced to ± 1 where an extremum may lie	Lemma S1.3	yes
$\ln 2, \pi$	exact rationals via atanh(1/3) and Machin's formula, converted outward	exact	through exp, log
Γ	Spouge's formula, $a = 20$, $z > 0.4$; the published bound [99], tenfold, enclosed; wide boxes by log-convexity	Lemma S1.4	yes
$\ln\Gamma, \psi, \psi'$	the argument lifted above 20 by the recurrences; Binet's series to $m = 8$; the first omitted term as bound; boxes by monotonicity, across the minimum by convexity	Lemma S1.5	yes
$P(a, z)$	series with a geometric tail; the prefactor in logarithms; boxes by monotonicity in both arguments	Lemma S1.6	yes
J_ν	power series by exact term ratios, 40 terms, a certified ratio bound	Lemma S1.7	yes
erf, erfc, Φ	erf by its alternating series for $\lvert z\rvert \le 2$, the next term times $2/\sqrt{\pi}$ as bound; erfc for $x \ge 1$ by Laplace's continued fraction [100, 7.9.2], consecutive convergents bracketing the value [101]	Lemma S1.8	yes
Φ^{-1}, P^{-1}	bisection that moves an end only on a certified sign	monotone	yes
tanh	$(e^{2x}-1)/(e^{2x}+1)$ from the exp enclosure, $[1 - 4\cdot 10^{-17}, 1]$ beyond $x = 20$; odd and increasing, a box by its ends	from Lemma S1.1	yes
Sum2	the compensated sum with the bound of Eq. (3) added outward	Ogita et al. [69, Prop. 4.5]	yes
the platform's cos, sin (older helper)	the platform's value padded by a derived margin; not proved; six callers, scheduled for retirement (Section 4.5)	none	yes

References

[1] C. Toledo, cert-machine: independent exact certification of machine-generated mathematics, The repository archive: code, records and reports. The concept DOI resolves to the latest version; the version this paper was built from is named on its title page (2026). doi:10.5281/zenodo.22225860.

[2] C. Toledo, A decision procedure for machine-proposed engineering numbers: three-valued verdicts over a declared uncertainty envelope, with witnesses and the threshold that decides, Archived with the repository, doi:10.5281/zenodo.22225860. Draft (2026).
URL https://carlostoledo.co/paper/p2-decision-procedure.pdf

[3] C. Toledo, Certified extreme-value analysis of significant wave height: maximum-likelihood fits and return levels over a public global hindcast, and environmental contours decided exactly, Archived with the repository, doi:10.5281/zenodo.22225860. Draft v0.2 (2026).
URL https://carlostoledo.co/paper/p1-return-levels.pdf

[4] G. Raayoni, S. Gottlieb, Y. Manor, G. Pisha, Y. Harris, U. Mendlovic, D. Haviv, Y. Hadad, I. Kaminer, Generating conjectures on fundamental constants with the Ramanujan Machine, Nature 590 (2021) 67–73. doi:10.1038/s41586-021-03229-4.

[5] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, A. Fawzi, Mathematical discoveries from program search with large language models, Nature 625 (2024) 468–475. doi:10.1038/s41586-023-06924-6.

[6] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, M. Balog, AlphaEvolve: a coding agent for scientific and algorithmic discovery (2025). arXiv:2506.13131.

[7] B. Georgiev, J. Gómez-Serrano, T. Tao, A. Z. Wagner, Mathematical exploration and discovery at scale (2025). arXiv:2511.02864.

[8] A. Z. Wagner, Constructions in combinatorics via neural networks (2021). arXiv:2104.14516.

[9] A. Davies, P. Veličković, L. Buesing, S. Blackwell, D. Zheng, N. Tomašev, R. Tanburn, P. Battaglia, C. Blundell, A. Juhász, M. Lackenby, G. Williamson, D. Hassabis, P. Kohli, Advancing mathematics by guiding human intuition with AI, Nature 600 (2021) 70–74. doi:10.1038/s41586-021-04086-x.

[10] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz, D. Silver, D. Hassabis, P. Kohli, Discovering faster matrix multiplication algorithms with reinforcement learning, Nature 610 (2022) 47–53. doi:10.1038/s41586-022-05172-4.

[11] S. Yang, R. Zhao, J. Wu, Y. Wang, F. Chen, K. Zhu, W. Li, Z. Li, Y. Li, S. Hong, Y. Pan, J. Yang, T. Deng, J. Mi, H. Chen, Y. Yang, Large-scale autonomous discovery of kissing number constructions (2026). arXiv:2609.35051.

[12] OpenAI, NavierStokesAndEuler, Commits 8937a8f4 and f9e8bc5b (2026).
URL https://github.com/openai/NavierStokesAndEuler

[13] OpenAI, math: manuscripts and Lean challenges, Commit fd4aeeb2 (2026).
URL https://github.com/openai/math

[14] T. H. Trinh, Y. Wu, Q. V. Le, H. He, T. Luong, Solving olympiad geometry without human demonstrations, Nature 625 (2024) 476–482. doi:10.1038/s41586-023-06747-5.

[15] C. Toledo, Independent exact certification of machine-generated mathematics: the first register, Archived with the repository, doi:10.5281/zenodo.22225860. Draft (2026).
URL https://carlostoledo.co/paper/register.pdf

[16] A. Bastounis, F. Circelli, A. C. Hansen, Navier–Stokes lost in translation: why Lean verification of AI autoformalisation does not guarantee correct natural language proofs (2026). arXiv:2610.08144.

[17] R. E. Moore, Methods and Applications of Interval Analysis, SIAM, 1979. doi:10.1137/1.9781611970906.

[18] A. Neumaier, Interval Methods for Systems of Equations, Cambridge University Press, Cambridge, 1990. doi:10.1017/CBO9780511526473.

[19] IEEE, IEEE standard for interval arithmetic, IEEE Std 1788-2015 (2015). doi:10.1109/IEEESTD.2015.7140721.

[20] R. Krawczyk, Newton-Algorithmen zur Bestimmung von Nullstellen mit Fehlerschranken, Computing 4 (1969) 187–201. doi:10.1007/BF02234767.

[21] S. M. Rump, Verification methods: rigorous results using floating-point arithmetic, Acta Numerica 19 (2010) 287–449. doi:10.1017/S096249291000005X.

[22] S. M. Rump, INTLAB — INTerval LABoratory, in: T. Csendes (Ed.), Developments in Reliable Computing, Kluwer, Dordrecht, 1999, pp. 77–104. doi:10.1007/978-94-017-1247-7_7.

[23] F. Johansson, Arb: efficient arbitrary-precision midpoint-radius interval arithmetic, IEEE Transactions on Computers 66 (8) (2017) 1281–1292. doi:10.1109/TC.2017.2690633.

[24] E. Hansen, G. W. Walster, Global Optimization Using Interval Analysis, 2nd Edition, Marcel Dekker, 2004. doi:10.1201/9780203026922.

[25] R. B. Kearfott, Rigorous Global Search: Continuous Problems, Kluwer, 1996. doi:10.1007/978-1-4757-2495-0.

[26] S. Fortune, C. J. Van Wyk, Static analysis yields efficient exact integer arithmetic for computational geometry, ACM Transactions on Graphics 15 (3) (1996) 223–248. doi:10.1145/231731.231735.

[27] J. R. Shewchuk, Adaptive precision floating-point arithmetic and fast robust geometric predicates, Discrete & Computational Geometry 18 (3) (1997) 305–363. doi:10.1007/PL00009321.

[28] C. K. Yap, Towards exact geometric computation, Computational Geometry 7 (1–2) (1997) 3–23. doi:10.1016/0925-7721(95)00040-2.

[29] S. Gao, J. Avigad, E. M. Clarke, δ-complete decision procedures for satisfiability over the reals, in: Automated Reasoning (IJCAR 2012), Vol. 7364 of Lecture Notes in Computer Science, 2012, pp. 286–300. doi:10.1007/978-3-642-31365-3_23.

[30] S. Gao, S. Kong, E. M. Clarke, dReal: An SMT solver for nonlinear theories over the reals, in: Automated Deduction (CADE-24), Vol. 7898 of Lecture Notes in Computer Science, 2013, pp. 208–214. doi:10.1007/978-3-642-38574-2_14.

[31] A. Solovyev, T. C. Hales, Formal verification of nonlinear inequalities with Taylor interval approximations, in: NASA Formal Methods (NFM 2013), Vol. 7871 of Lecture Notes in Computer Science, Springer, 2013, pp. 383–397. doi:10.1007/978-3-642-38088-4_26.

[32] É. Martin-Dorel, G. Melquiond, Proving tight bounds on univariate expressions with elementary functions in Coq, Journal of Automated Reasoning 57 (3) (2016) 187–217. doi:10.1007/s10817-015-9350-4.

[33] A. Mahboubi, G. Melquiond, T. Sibut-Pinote, Formally verified approximations of definite integrals, Journal of Automated Reasoning 62 (2019) 281–300. doi:10.1007/s10817-018-9463-7.

[34] W. L. Oberkampf, C. J. Roy, Verification and Validation in Scientific Computing, Cambridge University Press, Cambridge, 2010. doi:10.1017/CBO9780511760396.

[35] C. Toledo, Guaranteed 4D seismic detectability of gas and CO_2 injection fronts in a pre-salt-type carbonate: a decision over a declared petro-elastic envelope, Archived with the repository, doi:10.5281/zenodo.22225860. Draft (2026).
URL https://carlostoledo.co/paper/p3-4d-detectability.pdf

[36] R. E. Moore, R. B. Kearfott, M. J. Cloud, Introduction to Interval Analysis, SIAM, Philadelphia, 2009. doi:10.1137/1.9780898717716.

[37] N. Revol, Introduction to the IEEE 1788-2015 standard for interval arithmetic, in: Numerical Software Verification (NSV 2017), Vol. 10381 of Lecture Notes in Computer Science, Springer, 2017, pp. 14–21. doi:10.1007/978-3-319-63501-9_2.

[38] L. Benet, L. Ferranti, N. Revol, A framework to test interval arithmetic libraries and their IEEE 1788-2015 compliance, Concurrency and Computation: Practice and Experience 36 (3) (2024) e7856. doi:10.1002/cpe.7856.

[39] S. M. Rump, Interval arithmetic with fixed rounding mode, Nonlinear Theory and Its Applications, IEICE 7 (3) (2016) 362–373. doi:10.1587/nolta.7.362.

[40] A. Neumaier, Complete search in continuous global optimization and constraint satisfaction, Acta Numerica 13 (2004) 271–369. doi:10.1017/S0962492904000194.

[41] S. M. Rump, Mathematically rigorous global optimization in floating-point arithmetic, Optimization Methods and Software 33 (4–6) (2018) 771–798. doi:10.1080/10556788.2018.1435649.

[42] W. Tucker, A rigorous ODE solver and Smale's 14th problem, Foundations of Computational Mathematics 2 (2002) 53–117. doi:10.1007/s002080010018.

[43] F. Immler, A verified ODE solver and the Lorenz attractor, Journal of Automated Reasoning 61 (2018) 73–111. doi:10.1007/s10817-017-9448-y.

[44] T. Hales, M. Adams, G. Bauer, T. D. Dang, J. Harrison, L. T. Hoang, C. Kaliszyk, V. Magron, S. McLaughlin, T. T. Nguyen, Q. T. Nguyen, T. Nipkow, S. Obua, J. Pleso, J. Rute, A. Solovyev, T. H. A. Ta, N. T. Tran, T. D. Trieu, J. Urban, K. Vu, R. Zumkeller, A formal proof of the Kepler conjecture, Forum of Mathematics, Pi 5 (2017) e2. doi:10.1017/fmp.2017.1.

[45] J. Gómez-Serrano, Computer-assisted proofs in PDE: a survey, SeMA Journal 76 (2019) 459–484. doi:10.1007/s40324-019-00186-x.

[46] M. T. Nakao, M. Plum, Y. Watanabe, Numerical Verification Methods and Computer-Assisted Proofs for Partial Differential Equations, Springer, 2019. doi:10.1007/978-981-13-7669-6.

[47] A. Tarski, A decision method for elementary algebra and geometry, in: B. F. Caviness, J. R. Johnson (Eds.), Quantifier Elimination and Cylindrical Algebraic Decomposition, Springer, 1998, pp. 24–84, reprint of the 1951 RAND report R-109. doi:10.1007/978-3-7091-9459-1_3.

[48] D. Richardson, Some undecidable problems involving elementary functions of a real variable, Journal of Symbolic Logic 33 (4) (1968) 514–520. doi:10.2307/2271358.

[49] U. Loup, K. Scheibler, F. Corzilius, E. Ábrahám, B. Becker, A symbiosis of interval constraint propagation and cylindrical algebraic decomposition, in: Automated Deduction – CADE-24, Vol. 7898 of Lecture Notes in Computer Science, Springer, 2013, pp. 193–207. doi:10.1007/978-3-642-38574-2_13.

[50] S. C. Kleene, On notation for ordinal numbers, The Journal of Symbolic Logic 3 (4) (1938) 150–155. doi:10.2307/2267778.

[51] R. M. McConnell, K. Mehlhorn, S. Näher, P. Schweitzer, Certifying algorithms, Computer Science Review 5 (2) (2011) 119–161. doi:10.1016/j.cosrev.2010.09.009.

[52] G. C. Necula, Proof-carrying code, in: Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, 1997, pp. 106–119. doi:10.1145/263699.263712.

[53] N. Wetzler, M. J. H. Heule, W. A. Hunt, DRAT-trim: Efficient checking and trimming using expressive clausal proofs, in: Theory and Applications of Satisfiability Testing – SAT 2014, Vol. 8561 of Lecture Notes in Computer Science, 2014, pp. 422–429. doi:10.1007/978-3-319-09284-3_31.

[54] Y. K. Tan, M. J. H. Heule, M. O. Myreen, cake_lpr: Verified propagation redundancy checking in CakeML, in: Tools and Algorithms for the Construction and

Analysis of Systems (TACAS 2021), Vol. 12651 of Lecture Notes in Computer Science, 2021, pp. 223–241. doi:10.1007/978-3-030-72013-1_12.

[55] P. A. Parrilo, Semidefinite programming relaxations for semialgebraic problems, Mathematical Programming 96 (2) (2003) 293–320. doi:10.1007/s10107-003-0387-5.

[56] H. Peyrl, P. A. Parrilo, Computing sum of squares decompositions with rational coefficients, Theoretical Computer Science 409 (2) (2008) 269–281. doi:10.1016/j.tcs.2008.09.025.

[57] V. Magron, X. Allamigeon, S. Gaubert, B. Werner, Formal proofs for nonlinear optimization, Journal of Formalized Reasoning 8 (1) (2015) 1–24. doi:10.6092/issn.1972-5787/4319.

[58] C. K. Chow, On optimum recognition error and reject tradeoff, IEEE Transactions on Information Theory 16 (1) (1970) 41–46. doi:10.1109/TIT.1970.1054406.

[59] R. El-Yaniv, Y. Wiener, On the foundations of noise-free selective classification, Journal of Machine Learning Research 11 (2010) 1605–1641.
URL https://jmlr.org/papers/v11/el-yaniv10a.html

[60] Y. Geifman, R. El-Yaniv, Selective classification for deep neural networks, Advances in Neural Information Processing Systems 30 (NeurIPS 2017) (2017). arXiv:1705.08500.

[61] V. Franc, D. Prusa, V. Voracek, Optimal strategies for reject option classifiers (2021). arXiv:2101.12523.

[62] W. Li, et al., Falsification-based verification of LLM-generated outputs (2026). arXiv:2607.16646.

[63] D. Scholz, Theoretical rate of convergence for interval inclusion functions, Journal of Global Optimization 53 (4) (2012) 749–767. doi:10.1007/s10898-011-9735-9.

[64] M. Berz, K. Makino, Verified integration of ODEs and flows using differential algebraic methods on high-order Taylor models, Reliable Computing 4 (4) (1998) 361–369. doi:10.1023/A:1024467732637.

[65] IEEE, IEEE standard for floating-point arithmetic, IEEE Std 754-2019 (2019). doi:10.1109/IEEESTD.2019.8766229.

[66] D. Goldberg, What every computer scientist should know about floating-point arithmetic, ACM Computing Surveys 23 (1) (1991) 5–48. doi:10.1145/103162.103163.

[67] J.-M. Muller, N. Brunie, F. de Dinechin, C.-P. Jeannerod, M. Joldes, V. Lefèvre, G. Melquiond, N. Revol, S. Torres, Handbook of Floating-Point Arithmetic, 2nd Edition, Birkhäuser, 2018. doi:10.1007/978-3-319-76526-6.

[68] S. M. Rump, P. Zimmermann, S. Boldo, G. Melquiond, Computing predecessor and successor in rounding to nearest, BIT Numerical Mathematics 49 (2) (2009) 419–431. doi:10.1007/s10543-009-0218-z.

[69] T. Ogita, S. M. Rump, S. Oishi, Accurate sum and dot product, SIAM Journal on Scientific Computing 26 (6) (2005) 1955–1988. doi:10.1137/030601818.

[70] J. B. van den Berg, J.-P. Lessard, Rigorous numerics in dynamics, Notices of the American Mathematical Society 62 (9) (2015) 1057–1061. doi:10.1090/noti1276.

[71] Lean FRO, Comparator, Commit d03acab1 (2026).
URL https://github.com/leanprover/comparator

[72] ammkrn, nanoda_lib: a type checker for Lean 4 export files, Commit 3a240721, with the patch of issue #44 (2026).
URL https://github.com/ammkrn/nanoda_lib

[73] C. J. Clopper, E. S. Pearson, The use of confidence or fiducial limits illustrated in the case of the binomial, Biometrika 26 (4) (1934) 404–413. doi:10.1093/biomet/26.4.404.

[74] J. A. Hanley, A. Lippman-Hand, If nothing goes wrong, is everything all right? Interpreting zero numerators, JAMA 249 (13) (1983) 1743–1745. doi:10.1001/jama.1983.03330370053031.

[75] D. Ratz, T. Csendes, On the selection of subdivision directions in interval branch-and-bound methods for global optimization, Journal of Global Optimization 7 (2) (1995) 183–207. doi:10.1007/BF01097060.

[76] D. Troxell, et al., Stress-testing neural network verifiers with provably robust instances (2026). arXiv:2605.17153.

[77] T. Tao, et al., A collection of optimization problems in mathematics, The optimization-constants registry and its contributing guide; read at commits 2c1968cd (2026-09-26), ec37689f and 46dbf036 (2026-10-18), pinned in the repository (2026).
URL https://github.com/teorth/optimizationproblems

[78] R. A. DeMillo, R. J. Lipton, F. G. Sayward, Hints on test data selection: help for the practicing programmer, Computer 11 (4) (1978) 34–41. doi:10.1109/C-M.1978.218136.

[79] Y. Jia, M. Harman, An analysis and survey of the development of mutation testing, IEEE Transactions on Software Engineering 37 (5) (2011) 649–678. doi:10.1109/TSE.2010.62.

[80] G. Petrović, M. Ivanković, State of mutation testing at Google, in: Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP 2018), ACM, 2018, pp. 163–171. doi:10.1145/3183519.3183521.

[81] D. Winterer, C. Zhang, Z. Su, On the unusual effectiveness of type-aware operator mutations for testing SMT solvers, Proceedings of the ACM on Programming Languages 4 (OOPSLA) (2020) 193. doi:10.1145/3428261.

[82] M. N. Mansur, M. Christakis, V. Wüstholz, F. Zhang, Detecting critical bugs in SMT solvers using blackbox mutational fuzzing, in: Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020), ACM, 2020, pp. 701–712. doi:10.1145/3368089.3409763.

[83] J. C. Knight, N. G. Leveson, An experimental evaluation of the assumption of independence in multiversion programming, IEEE Transactions on Software Engineering SE-12 (1) (1986) 96–109. doi:10.1109/TSE.1986.6312924.

[84] P. Nogueira, et al., A systematic methodology for evaluating failure independence of LLM-generated program variants (2026). arXiv:2607.02808.

[85] D. Ron, et al., N-version programming with coding agents (2026). arXiv:2606.20158.

[86] Oxelra-AI, Qiushi-Engine-Kissing-Number-Research: the complete point sets, Releases section-vectors-2026-10-11, configuration-vectors-2026-10-12 and lifting-vectors-2026-10-12, and issue #1; data CC BY 4.0 (2026).
URL https://github.com/Oxelra-AI/Qiushi-Engine-Kissing-Number-Research

[87] T. Tao, Drop the unverified asterisk on the C_{3b} record, The maintainer's comment of 2026-09-26 on pull request 176 to the optimization-constants registry, which another contributor opened under this title; the comment states the rule that the registry's contributing guide then carried (2026).
URL https://github.com/teorth/optimizationproblems/pull/176#issuecomment-5848109776

[88] rainrzk, erdos510-lambda4-audit: an independent re-certification of the $\lambda(4)$ argument with no shared code, Posted 2026-09-30 on teorth/erdosproblems issue #392 (2026).
URL https://github.com/rainrzk/erdos510-lambda4-audit

[89] S. Kong, A. Solar-Lezama, S. Gao, Delta-decision procedures for exists-forall problems over the reals, in: Computer Aided Verification (CAV 2018), Vol. 10982 of Lecture Notes in Computer Science, Springer, 2018, pp. 219–235. doi:10.1007/978-3-319-96142-2_15.

[90] S. Gao, S. Kong, E. M. Clarke, Proof generation from delta-decisions, in: 16th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC 2014), IEEE, 2014, pp. 156–163. doi:10.1109/SYNASC.2014.29.

[91] M. Carneiro, Lean4Lean: towards a verified typechecker for Lean, in Lean (2024). arXiv:2403.14064.

[92] M. Kallel, et al., Verification abundance, adjudication scarcity (2026). arXiv:2608.28997.

[93] N. P. Rougier, K. Hinsen, F. Alexandre, T. Arildsen, L. A. Barba, F. C. Y. Benureau, et al., Sustainable computational science: the ReScience initiative, PeerJ Computer Science 3 (2017) e142. doi:10.7717/peerj-cs.142.

[94] C. Lamb, S. Zacchiroli, Reproducible builds: Increasing the integrity of software supply chains, IEEE Software 39 (2) (2022) 62–70. doi:10.1109/MS.2021.3073045.

[95] R. Koomullil, et al., Proof-carrying certificates for LLM pipelines (2026). arXiv:2605.16407.

[96] M. Fasi, N. J. Higham, M. Mikaitis, S. Pranesh, Numerical behavior of NVIDIA tensor cores, PeerJ Computer Science 7 (2021) e330. doi:10.7717/peerj-cs.330.

[97] P. Blanchard, N. J. Higham, F. Lopez, T. Mary, S. Pranesh, Mixed precision block fused multiply-add: error analysis and application to GPU tensor cores, SIAM Journal on Scientific Computing 42 (3) (2020) C124–C141. doi:10.1137/19M1289546.

[98] N. J. Higham, T. Mary, Mixed precision algorithms in numerical linear algebra, Acta Numerica 31 (2022) 347–414. doi:10.1017/S0962492922000022.

[99] J. L. Spouge, Computation of the gamma, digamma, and trigamma functions, SIAM Journal on Numerical Analysis 31 (3) (1994) 931–944. doi:10.1137/0731050.

[100] NIST, NIST digital library of mathematical functions, Release 1.2.4 of 2025-03-15, F. W. J. Olver, A. B. Olde Daalhuis, D. W. Lozier, B. I. Schneider, R. F. Boisvert, C. W. Clark, B. R. Miller, B. V. Saunders, H. S. Cohl, and M. A. McClain, eds. (2025).
URL https://dlmf.nist.gov/

[101] A. Cuyt, V. B. Petersen, B. Verdonk, H. Waadeland, W. B. Jones, Handbook of Continued Fractions for Special Functions, Springer, 2008. doi:10.1007/978-1-4020-6949-9.